B.Tech. BCSE497J - Project-I

SELECTIVE COOKIE OUTSOURCING USING PRIVATE INFORMATION RETRIEVAL

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science & Engineering

by

22BCE0756 APOORVA KARNWAL

22BCE2232 SHIVESH KAUSHIK

Under the Supervision of

KATHIRAVAN SRINIVASAN

Professor Grade 1

School of Computer Science and Engineering (SCOPE)

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

November 2025

DECLARATION

I hereby declare that the project entitled Selective Cookie Outsourcing using Private Information Retrieval submitted by me, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT is a record of bonafide work carried out by me under the supervision of Prof. / Dr. Kathiravan Srinivasan.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date : 05/11/2025

Signature of the Candidate

CERTIFICATE

This is to certify that theproject entitled Selective Cookie Outsourcing using Private Information Retrieval submitted by Apoorva Karnwal (22BCE0756), Shivesh Kaushik (22BCE2232) **School of Computer Science and Engineering**, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by him / her under my supervision during Fall Semester 2024-2025, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute orany other institute or university. The project fulfills the requirements and regulations ofthe University and in my opinion meets the necessary standards for submission.

Place : Vellore
Date : 05/11/2025

Signature of the Guide

Examiner(s)

Dr. Boominathan P
B. Tech – Computer Science & Engineering

ACKNOWLEDGEMENTS

I am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering (SCOPE), for his unwavering support and encouragement. His leadership and vision have greatly inspired me to strive for excellence. The Dean's dedication to academic excellence and innovation has been a constant source of motivation for me. I appreciate his efforts in creating an environment that nurtures creativity and critical thinking.

I express my profound appreciation to Dr. Boominathan P, the Head of the Department of Software Systems , for his insightful guidance and continuous support. His expertise and advice have been crucial in shaping the direction of my project. The Head of Department's commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. His constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project guide, Dr. Kathiravan Srinivasan, for his dedicated mentorship and invaluable feedback. His patience, knowledge, and encouragement have been pivotal in the successful completion of this project. My supervisor's willingness to share his expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. His support has not only contributed to the success of this project but has also enriched my overall academic experience.

Thank you all for your contributions and support.

Apoorva Karnwal, Shivesh Kaushik

TABLE OF CONTENTS

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Description
AES	Advanced Encryption Standard
API	Application Programming Interface
CBM	CatBoost Model
CMP	Consent Management Platform
CPIR	Computational Private Information Retrieval
GDPR	General Data Protection Regulation
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
LGBM	Light Gradient Boosting Machine
ML	Machine Learning
ONNX	Open Neural Network Exchange
PIR	Private Information Retrieval
SDK	Software Development Kit
UI	User Interface
URL	Uniform Resource Locator
WASM	WebAssembly
XGB	Extreme Gradient Boosting (XGBoost)

Symbols and Notations

F1	F1-score representing the harmonic mean of precision and recall.
K	Symmetric encryption key used for securing cookie data during storage

ABSTRACT

Browser cookies have become a fundamental component of modern web architecture, enabling essential features such as user authentication, session management, personalization, and analytics. While these functions significantly improve user experience, they also create privacy vulnerabilities by exposing individuals to behavioral profiling, cross-site tracking, and potential regulatory violations. The increasing reliance of websites on third-party cookies for targeted advertising and data collection aggravates this issue, leaving users with limited control over how their personal information is collected, stored, and shared. Existing solutions, such as cookie consent banners and Consent Management Platforms (CMPs), primarily address regulatory compliance rather than enforcing strict security controls. Furthermore, there is a growing demand for mechanisms that balance usability, privacy, and performance in cookie management.

This research study proposes a novel hybrid framework that integrates machine learning (ML) classification techniques with Private Information Retrieval (PIR) protocols to provide an effective and practical approach for cookie privacy. To build the dataset cookies are scraped from websites and then paired with their category labels obtained from the CMPs. From each cookie we extract attributes such as HttpOnly, Secure, and SameSite flags, along with other contextual features categorizing cookies into essential, functional, analytics, or marketing types. These features serve as inputs to supervised ML models, including XGBoost, LightGBM and CatBoost, which are trained to identify cookie category with high accuracy.

To address storage and retrieval concerns, the study incorporates a PIR mechanism that securely outsources sensitive cookies to a cloud/local environment. By utilizing homomorphic encryption, the PIR protocol ensures that users can retrieve their data without revealing access patterns or compromising security. Furthermore, recent advances in PIR efficiency, particularly low-communication and high-rate protocols like the PIR, are leveraged to minimize the latency typically associated with cryptographic methods. The approach proposed in this study makes three key contributions. First, it demonstrates cookie classification using attribute-level and contextual features. Second, it introduces a selective storage mechanism that applies PIR only where necessary, significantly reducing overhead. Third, it offers a scalable and deployable architecture for integrating technical privacy protections within browsers. The expected outcome is a framework that ensures user privacy while maintaining seamless web functionality, making it a promising candidate for adoption in privacy-conscious browser ecosystem

CHAPTER 1

1.1 INTRODUCTION

The rise of web technologies has reshaped how users access services and information. Central to this evolution is the HTTP cookie, or browser cookie, a lightweight data structure that stores stateful information in an otherwise stateless web environment. Initially, cookies were designed as a convenience for session management, but they have since evolved into a powerful tool that supports authentication, personalization, analytics, and large-scale targeted advertising. While this evolution enables seamless user experiences and improves usability, it has also introduced complex privacy and security concerns.

Cookies often persist beyond a single browsing session, are accessible across domains and often contain sensitive user information such as session tokens. Cookies can be exploited for behavioral profiling, cross-site tracking, masquerading or even session hijacking [1]. Modern regulatory frameworks such as the General Data Protection Regulation (GDPR) [6] in the European Union and the California Consumer Privacy Act (CCPA) in the United States, attempt to mitigate these risks by mandating explicit consent from the users for data collection and cookie acceptance. However, the compliance mechanisms such as cookie banners or Consent Management Platforms (CMPs) frequently prove inadequate and fail to implement the user consent, as studies show that many websites mislabel cookies, ignore user preferences, or employ manipulative "dark patterns" [1,5,6] that steer users toward acceptance.

In this context, the need arises for solutions that go beyond regulatory compliance and provide stronger, practical privacy protections for cookies. Simply blocking cookies is not feasible as it may interfere with website functionality and usability. Recent progress in machine learning (ML) [3,4] for automatic cookie classification, along with advances in Private Information Retrieval (PIR) protocols [8], creates an opportunity for a more balanced approach. Such a system can separate sensitive cookies from non-sensitive ones, protecting only the critical data with cryptographic methods while keeping performance costs low and reducing latency.

This research proposes a framework combining ML-based cookie classification with selective PIR storage for privacy-preserving cookie management. The ML models use cookie attributes and HttpOnly, Secure, and SameSite flags to classify cookies into meaningful categories. Based on these results, only sensitive cookies are outsourced via PIR protocols, ensuring secure retrieval without revealing access patterns. This combined strategy enhances privacy, maintains usability, and reduces the overhead of outsourcing all cookies to the secure server.

1.1 Background

The effectiveness of any privacy-related solutions for cookies requires a deeper understanding of how cookies function, the various attributes and properties of an http cookies, the legal frameworks governing them, and the limitations of current enforcement mechanisms. This section provides the technical and contextual foundantion for the proposed work, beginning with the structure and attributes of cookies, then discussing data protection regulations and consent platforms, and finally highlighting recent advances in machine learning on the topic of cookie classification and PIR that motivate a combined approach.

1.1.1 Http Cookies

Http Cookies, also known as browser cookies, are small pieces of data sent from a web server to the user's browser. Cookies allow web applications to store limited amounts of information about user interactions, making web experiences more seamless and interactive. Cookies enable websites to recognize returning users, maintain session continuity, personalize content, and perform analytics or targeted advertising [3]. This capability to associate requests with a specific user session is fundamental to how the modern web functions.

Cookies serve crucial purposes that enhance the overall user experience and provide websites with valuable data.The primary functions of cookies are as follows: (1) Session management uses cookies to track login status, shopping cart contents, or user progress in web applications. (2) Personalization allows websites to remember user preferences, such as language selection or theme settings (dark or light mode). (3) Tracking and analytics use cookies to monitor browsing behaviour, measure engagement, and support targeted advertising. While cookies were historically used for general client-side data storage, modern alternatives like localStorage, sessionStorage, and IndexedDB are now recommended for larger or non-sensitive data.

The lifecycle of cookie, creation, updation and deletion, is managed through HTTP headers. Server creates cookies by including Set-Cookie header in its response. The user's browser then stores this data and automatically includes it in subsequent requests to the same domain via the Cookie header. Cookies can be either session cookies, which are temporary and are deleted when the browsing session ends, or persistent cookies, which remain on the user's device until a specified expiration date or duration. Developers can update a cookie by sending a Set-Cookie header with the same name but a new value. While cookies can also be managed via JavaScript using Document.cookie, access may be restricted by security attributes like HttpOnly. Improper handling, such as the use of "zombie cookies" that are recreated after deletion, can lead to privacy violations and non-compliance with regulations.

Cookies include several attributes that control their behavior and security. The Domain and Path attributes determine which URLs will receive the cookie. The Secure attribute ensures cookies are transmitted only over HTTPS, preventing interception on unsecured networks. The HttpOnly attribute blocks JavaScript access, reducing exposure to cross-site scripting (XSS) attacks. The SameSite attribute helps prevent cross-site request forgery (CSRF) and limits third-party cookie exposure, with values such as Strict, Lax, or None defining the scope of cross-site access. Additionally, cookie prefixes like __Secure- and __Host- enforce stricter security rules, ensuring cookies are set on secure origins with limited scope and reducing risks from session fixation or subdomain misuse.

1.1.2 Data Protection Regulations

The General Data Protection Regulation (GDPR), in force across the EU since 2018, is one of the strictest global privacy laws [5]. It mandates explicit, informed consent before personal data, including cookies, can be processed. Non-compliance carries heavy penalties of up to €20 million or 4% of worldwide turnover, compelling websites to implement consent banners and stricter cookie handling.

The California Consumer Privacy Act (CCPA), effective from 2020, gives residents of California

rights to know, opt out of, and request deletion of personal data. Cookies are covered under its definition of personal information. Unlike GDPR's opt-in approach, CCPA follows an opt-out model, requiring users to take action if they do not want their data shared, especially for advertising purposes.

Despite these frameworks, enforcement challenges remain. Across all three regions, compliance often relies on banners or CMPs that may be misleading, poorly implemented, or ignored by websites. Users, confronted with frequent pop-ups, often default to "accept all," [1,6] undermining the intended protections and leaving privacy risks unresolved.

1.1.3 Consent Management Platforms (CMPs)

Consent Management Platforms (CMPs) were developed to help website operators comply with regulations like GDPR and CCPA. CMPs automate the collection of user preferences and ensure these choices are communicated to third-party vendors. Well-known examples include OneTrust and Cookiebot.

CMPs offer several strengths. They reduce the compliance burden for website owners and provide standardized mechanisms, such as those defined by the IAB's Transparency and Consent Framework (TCF). By centralizing consent management, CMPs also simplify integration across multiple advertising and analytics platforms.

However, CMPs also face weaknesses. Many uses dark patterns pressuring users into accepting all cookies [1,5,6]. Others fail to respect explicit rejection signals, continuing to set non-essential cookies despite a user's choice. In practice, adoption of CMPs remains limited, with studies reporting that fewer than 3% of top websites deploy them effectively [5]. As a result, CMPs are better viewed as compliance aids rather than robust privacy enforcement systems.

1.1.4 Machine Learning for Cookie Classification

Machine learning (ML) provides a systematic approach for automating cookie classification. By analyzing cookie attributes such as HttpOnly, Secure, and SameSite flags, along with the contextual metadata, ML models can predict the functional category of a cookie, whether, for example, between essential, functional, analytical, and tracking cookies.

Previous research has demonstrated the feasibility of this approach [3,4]. For instance, the CookieBlock project successfully applied gradient-boosting classifiers such as XGBoost and CatBoost to classify cookies with higher accuracy than public repositories like Cookiepedia [6]. These models have proven capable of recognizing subtle attribute patterns that humans or rule-based systems may overlook.

That said, ML-based classification has clear limitations. While it can effectively detect and categorize cookies, classification alone does not enforce security or privacy policies. Without an accompanying mechanism to control storage and retrieval, sensitive cookies remain vulnerable to misuse once identified [7].

1.1.5 PIR - Private Information Retrieval

Private Information Retrieval (PIR) is a cryptographic technique that enables a client to query a database and retrieve a record without revealing which record was accessed. This ensures that the

server storing the data cannot infer the user's interests or activities from their requests.

Historically, PIR faced significant performance challenges. Early protocols imposed heavy computational and communication costs, making them impractical for real-time applications such as web browsing [8]. As a result, PIR was mostly limited to theoretical discussions.

Recent advances, however, have transformed the landscape. Developments in homomorphic encryption and optimized PIR schemes, including high-rate protocols proposed in 2023 [9], have reduced latency and bandwidth requirements. These improvements make PIR a practical option for securing sensitive web interactions. Within the scope of this project, PIR provides a means to outsource sensitive cookies to a secure storage environment, ensuring they can be retrieved when needed without exposing patterns or compromising confidentiality [8, 9].

1.2 Motivations

Cookies are vital for authentication, personalization, and analytics but are often misused for tracking, profiling, and cross-site surveillance, creating serious privacy risks. Existing compliance measures like consent banners and CMPs are inadequate, as they are frequently misimplemented, misleading, or ignored, leaving users vulnerable and eroding trust.

Completely blocking cookies is impractical since it disrupts essential web functionality. Hence, there is a need for solutions that balance privacy with usability and performance. Advances in machine learning (ML) for cookie classification and cryptographic methods like Private Information Retrieval (PIR) provide an opportunity to build a scalable framework that offers strong technical privacy protections while preserving seamless web experiences.

1.3 Scope of the Project

This project develops a hybrid framework for cookie privacy by combining machine learning (ML) classification with Private Information Retrieval (PIR). Cookies are collected with attributes like HttpOnly, Secure, SameSite, and expiry, then classified into essential, functional, analytical, or tracking types using gradient-boosting models such as XGBoost and CatBoost. Sensitive cookies are outsourced to secure storage via efficient PIR protocols, while non-sensitive ones remain local to reduce overhead. The framework aims to balance privacy, usability, and performance, and will be evaluated for accuracy, latency, efficiency, and scalability to support integration into browsers or privacy-focused platforms.

CHAPTER 2

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Berens et al. (2024) provide how using consent banners on website to store cookies is fragile. Through the study the authors show that visual layout of accept/reject banner of cookies strongly influence whether people accept cookies. Many users click “accept” out of habit, fear of site breakage or time, indicating most do not read the banner text. Crucially the paper finds that 70% of the 376 sites still set non-essential cookies even when the user refuses them. This demonstrates how consent UI is quite ineffective and how the cookies are being stored even after refusal.

Bouhoula et al. (USENIX Security,2023) scale the problem up by building a crawler that interacts with cookie banners across 97,000+ targeted websites and use NLP to infer purpose of consent. The study confirms that there is a widespread non-compliance, and the websites choose to not show reject option . The paper also discusses openWPM based crawling with interactive element detection.

Hu et al. (WebSci 2021) introduce CookieMonster, an efficient ML system that classifies cookies using lexical features of cookie names. Their approach shows surprisingly strong performance: simple tokenization and lightweight classifiers produced high F1 scores. The paper also notes limitations-cookie names can be misleading and classification coverage depends on the underlying labeled database. For our work, CookieMonster suggests that fast, name-based classification can be a valuable first stage.

Munir et al. (CCS 2023) address a growing evasion technique: trackers moving to first-party cookies as browsers clamp down on third-party cookies. Their study build a graph-based approach with JavaScript execution traces, network traffic, and storage access to detect first party tracking cookies. This paper tells us that lexical analysis alone is not enough behavioral and network features matter, especially as tracking strategies evolve as well as the cookie attributes.

Sivan-Sevilla et al. (2025) analyze “cookie-less” first-party identity solutions (universal IDs, onboarding approaches) can also result in surveillance risks. This paper helps frame the adversarial landscape: even if browsers remove third-party cookies, tracking shifts and persistence remains a concern. This leads to the need of more secure environment and prevention of user tracking .

Vithana et al. (2023) provide a comprehensive PIR survey that lays out the practicality trade-offs between multi-server information-theoretic PIR and single-server computational PIR (often using homomorphic encryption), as well as advanced variants like batched PIR and PIR with side information. The survey is clear about the main bottleneck likes communication and computation costs which directly indicates to choose which PIR method for our case.

Luo and Wang (2025) introduce *Faster Spiral*, an optimized variant of the well-known Spiral PIR protocol, designed to improve throughput without sacrificing its key strengths of low communication and high retrieval rate**.** Implemented in C++, these optimizations deliver around 1.7 × faster performance compared to the original Spiral, while maintaining small query sizes (14 KB) and high throughput. The paper shows that PIR can be used for more practical scenarios where there are constraints.

2.2 Research Gaps

Across all the above papers we derive that the user-facing consent mechanisms are ineffective, auditing shows systematic non-compliance. While banners and Consent Management Platforms (CMPs) are meant to empower users, they are routinely undermined by manipulative design patterns and backend practices that continue to set non-essential cookies even after explicit rejection. The machine learning based cookies classification offers accuracy but and do not provide a pathway for protecting sensitive cookies once identified. Furthermore, cryptographic tools like Private Information Retrieval provide strong theoretical privacy but lack practical implementations.

What missing is a combined approach that can reliably identify genuine privacy sensitive cookies using lexical, attribute-based and contextual signals and securely store them in a practical manner. Our study addresses this gap through a hybrid machine learning + selective PIR architecture. Our study leverages practical classification models (like CookieGraph and CookieMonster), creating labelled dataset via automated CMP parsing (Bouhoula et al.) and applying PIR to sensitive cookies (building on Vithana et al. and Luo et al.).

2.3 Objectives

The primary objective of this research is to design and evaluate a hybrid framework that enhances privacy in cookie management by combining machine learning (ML)-based classification with Private Information Retrieval (PIR) protocols. To achieve this, cookies are collected from real-world websites and enriched with technical attributes such as HttpOnly, Secure, SameSite, and expiry, as well as contextual metadata derived from consent management platforms. These features provide the foundation for training supervised ML models, including XGBoost, LightGBM, and CatBoost, to categorize cookies into essential, functional, analytical, and tracking types. By systematically classifying cookies, the framework aims to distinguish those that are privacy-sensitive and require additional protections.

Beyond classification, the objective extends to designing a selective storage and retrieval mechanism that secures sensitive cookies through optimized PIR protocols while leaving non-sensitive cookies stored locally. This selective outsourcing ensures that cryptographic methods are applied only where necessary, reducing both latency and computational overhead. In doing so, the framework not only strengthens privacy guarantees but also preserves usability and performance. Ultimately, the research seeks to provide a deployable architecture that balances efficiency, scalability, and privacy, with the long-term goal of enabling integration into browsers or privacy-focused platforms for stronger technical protections against misuse of cookies.

2.4 Problem Statement

HTTP cookies are essential for web functionality, supporting authentication, session management, personalization, and analytics, but they have also become tools for tracking, profiling, and unauthorized data collection. Regulatory mechanisms like cookie banners and CMPs focus on compliance rather than on enforcement, with studies showing that a majority of websites still set non-essential cookies even after explicit rejection. Blocking cookies entirely disrupts website usability, while existing technical solutions fail to balance privacy with functionality, often applying one-size-fits-all protections that ignore the different roles of various cookie types.

The central challenge is the lack of an intelligent, practical system that can distinguish between essential and privacy-sensitive cookies at scale and apply selective protections without introducing prohibitive overhead. Machine learning offers accurate classification, but on its own cannot enforce security, while cryptographic methods like PIR ensure privacy but impose high performance costs if applied universally. This project addresses the gap by proposing a hybrid ML + PIR framework that automatically classifies cookies into functional categories and applies efficient PIR protocols only to sensitive cookies, ensuring privacy protection without sacrificing usability or performance.

2.5 Project Plan

Step 1: Data Collection

- Search for existing labeled cookie datasets for benchmarking
- Scrape a domain list of top websites from Cookiepedia
- Use OpenWPM integrated with Consent-O-Matic to detect CMPs (Consent Management Platforms) across domains
- Extract a list of domains with active CMPs
- Fetch cookie categorization labels directly from CMPs
- Crawl each website by accepting all cookies to obtain raw cookie datasets

Step 2: Data Preprocessing

- Remove duplicate cookies across domains
- Normalize domain names (e.g., ".google.com" → "google.com")
- Perform logical joins on cookie name and domain to link cookies with CMP-provided labels
- Convert boolean cookie attributes into binary values (true → 1, false → 0)
- Extract contextual features such as expiry time, first-party/third-party classification, and other metadata
- Encode categorical features (e.g., top-level domain, SameSite values) into numerical form for ML models

Step 3: Machine Learning Model Development

- Split the labeled dataset into training and testing subsets
- Train supervised ML models such as XGBoost, LightGBM, and CatBoost on cookie attributes and contextual features
- Compare model performance and select the best-performing classifier
- Evaluate using accuracy, precision, recall, and F1-score to ensure balanced performance

Step 4: PIR Integration

- Select the appropriate PIR protocol type: Computational PIR (single-server) or Information-Theoretic PIR (multi-server)

- Implement the chosen PIR scheme using homomorphic encryption or optimized PIR variants
- Test secure storage and retrieval of sensitive cookies
- Integrate PIR-based storage with ML outputs, applying PIR only to privacy-sensitive cookies
- Prototype integration into a browser extension or middleware for proof-of-concept deployment

Step 5: Evaluation and Validation

- Measure system performance in terms of classification accuracy, latency, and efficiency
- Evaluate network and bandwidth overhead of PIR retrieval
- Conduct privacy and security assessments to ensure compliance with GDPR and CCPA
- Compare results against baseline methods such as cookie blocking and CMP-only consent mechanisms
- Document implementation details, experimental results, and write the final report.

2.5.1 Gannt Chart

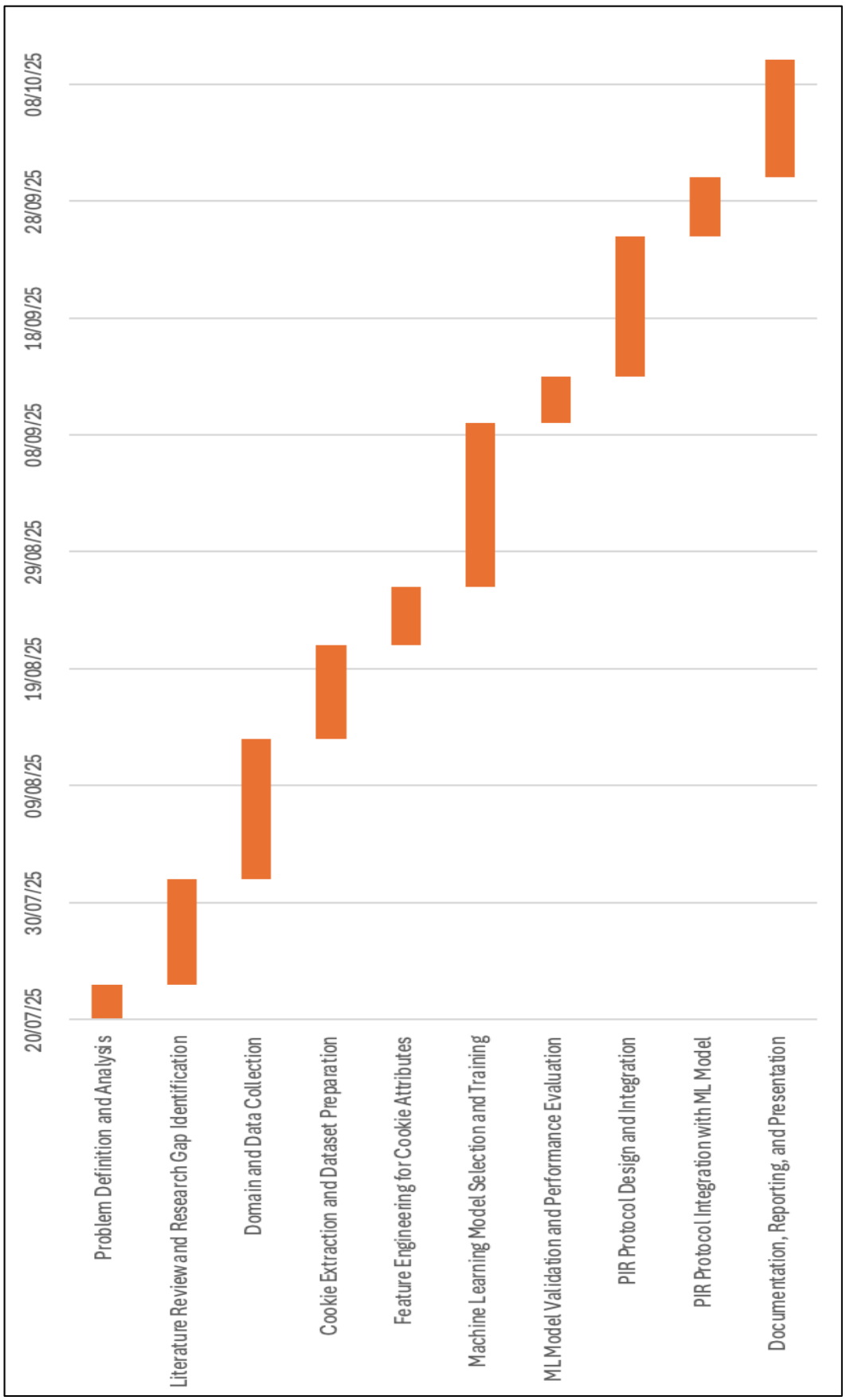

Fig 2.1 – Gannt Chart

2.5.2 Work Breakdown Structure

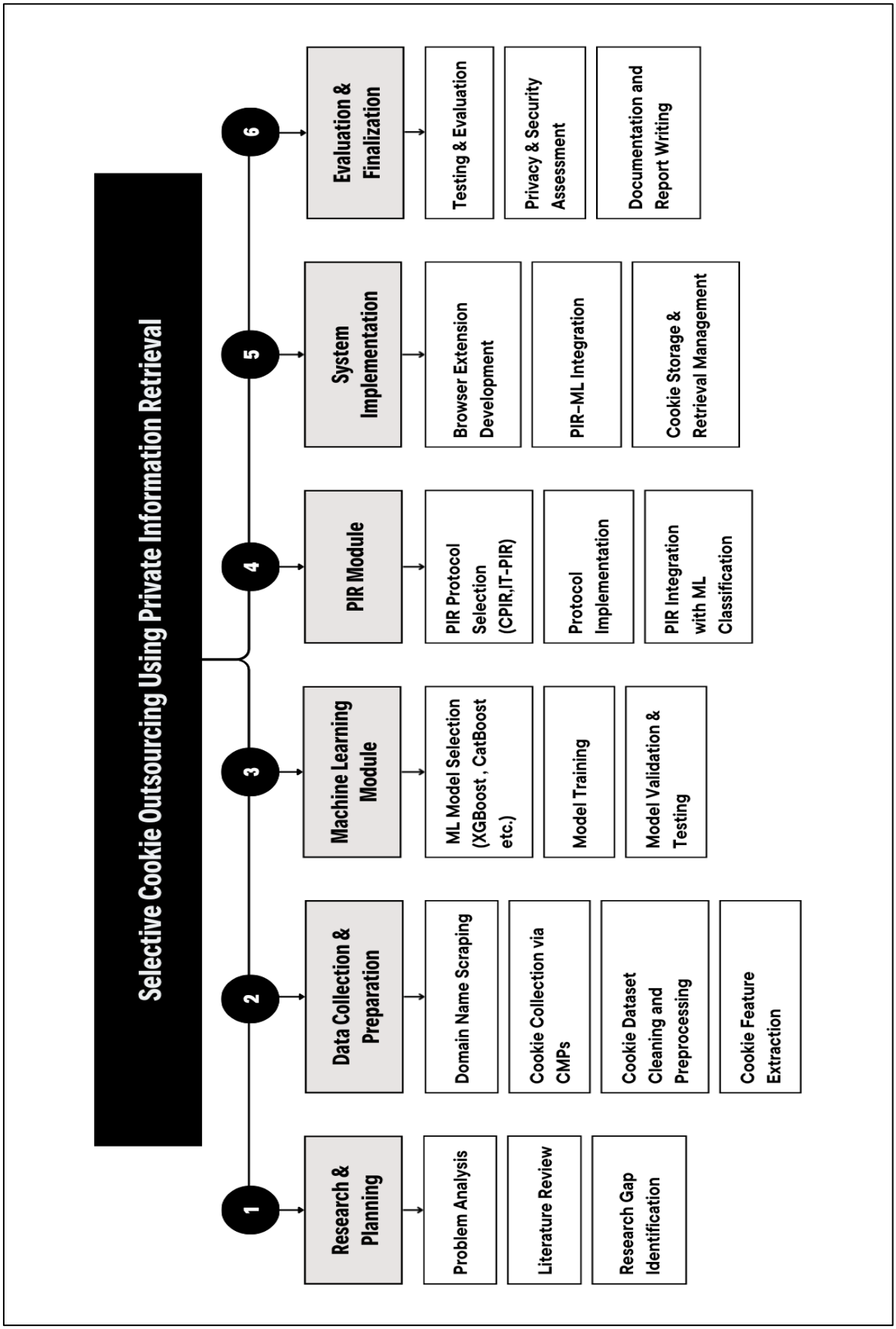

Fig 2.2 – Work Breakdown Structure

CHAPTER 3

3. TECHNICAL SPECIFICATION

3.1.1 Functional Requirements

- The system automatically crawls and collects cookies from real-world websites using OpenWPM or equivalent data collection tools.
- It extracts and preprocesses cookie attributes such as HttpOnly, Secure, SameSite, and expiry, along with contextual metadata derived from Consent Management Platforms (CMPs).
- Supervised Machine Learning models (XGBoost, LightGBM, CatBoost) are effectively trained to classify cookies into categories such as essential, functional, analytics, and marketing.
- Sensitive cookies identified by the ML models are stored using Private Information Retrieval (PIR) protocols to ensure privacy-preserving retrieval without exposing access patterns.
- The framework supports a modular and highly scalable architecture, enabling easy integration of cookie classification and PIR components into browsers or middleware environments.
- The system provides a visualization dashboard showing classification results, storage distribution, and privacy improvements.

3.1.2 Non-Functional Requirements

- Performance: The framework must process and classify cookies efficiently, maintaining low latency during classification and retrieval.
- Scalability: It should support large-scale web crawling and data processing across thousands of domains.
- Reliability: The system must handle dynamic websites, malformed cookies, and diverse CMP configurations without failure.
- Usability: The solution should integrate smoothly into browser extensions or middleware without affecting user experience.
- Security: PIR protocols and encryption methods must guarantee confidentiality and prevent inference of user access patterns.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The project leverages widely used and well-supported technologies such as Python 3.9+, OpenWPM, scikit-learn, and cryptography libraries. Existing machine learning frameworks like XGBoost and LightGBM simplify the model training and optimization process, while private information retrieval (PIR) can be implemented using modern homomorphic encryption techniques or optimized protocols such as PIR. The modular system design allows for seamless integration between the backend server, browser extension, and privacy layer, ensuring

compatibility with both local and cloud-based environments. Given the maturity of these tools and the availability of research-grade hardware and libraries, the system is technically feasible and sustainable for long-term use or further expansion.

3.2.2 Economic Feasibility

The implementation primarily depends on open-source tools and libraries, which significantly reduces overall project costs. Computational requirements are modest-standard personal computers or affordable cloud-based virtual machines are sufficient for training, testing, and running the models. Since no commercial APIs or paid licenses are involved, the financial investment remains minimal. The project's design also supports containerization through Docker, reducing deployment costs and simplifying scaling for future use. Therefore, the system is economically viable for academic research, prototype development, or small-scale commercial implementation.

3.2.3 Social Feasibility

The proposed framework aligns closely with increasing public concern for data privacy and ethical online practices. It empowers users to control how their cookies are stored and used, promoting transparency and accountability in digital interactions. By incorporating privacy-preserving methods such as encryption and selective storage, the system enhances user trust without compromising browsing convenience. Furthermore, it adheres to international data protection standards like GDPR and CCPA, reflecting a socially responsible approach to technology design. As a result, the project contributes positively to public digital welfare and encourages safer internet usage habits.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.1 – Hardware Specification

Component	Minimum Requirement	Recommended Specification
Processor	Intel i5 or equivalent	Intel i7 / AMD Ryzen 7
Ram	8 GB	16 GB
Storage	50 GB free space	100 GB SSD
Operating System	Windows 10 / Ubuntu 20.04+ / macOS 12+	Ubuntu 22.04 LTS / macOS 13+
Network	Stable broadband connection	High-speed network for crawling and training

3.3.2 Software Specification

- **Programming Languages:** Python 3.8+ (for model training), Python 3.11 (for server-side operations), JavaScript, HTML, and CSS (for the browser extension).
- **Data Collection Tools:** OpenWPM for large-scale web crawling and a custom browser extension for on-device cookie capture.
- **Data Handling Libraries:** pandas, numpy.
- **Machine Learning Libraries:** scikit-learn, XGBoost, LightGBM, and CatBoost for model training and evaluation.
- **Model Export and Inference:** ONNX, skl2onnx, onnxmltools, and ONNX Runtime Web for exporting and running lightweight models in the browser.
- **Backend Framework:** Flask with Flask-CORS for handling REST API requests and communication with the browser extension.
- **Privacy and Encryption:** PyCryptodome for prototype-level encryption, extendable to SealPIR or SimplePIR for secure private information retrieval.
- **Visualization Tools:** Web-based dashboard for real-time logs and performance metrics, with optional Jupyter Notebook for analytical visualization.
- **Storage System:** JSON-based cookie storage with Docker volume persistence for long-term data management.
- **Tools and Deployment:** Git for version control, Docker and Docker Compose for deployment, and optional Jupyter Notebook for experimentation and reporting

CHAPTER 4

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

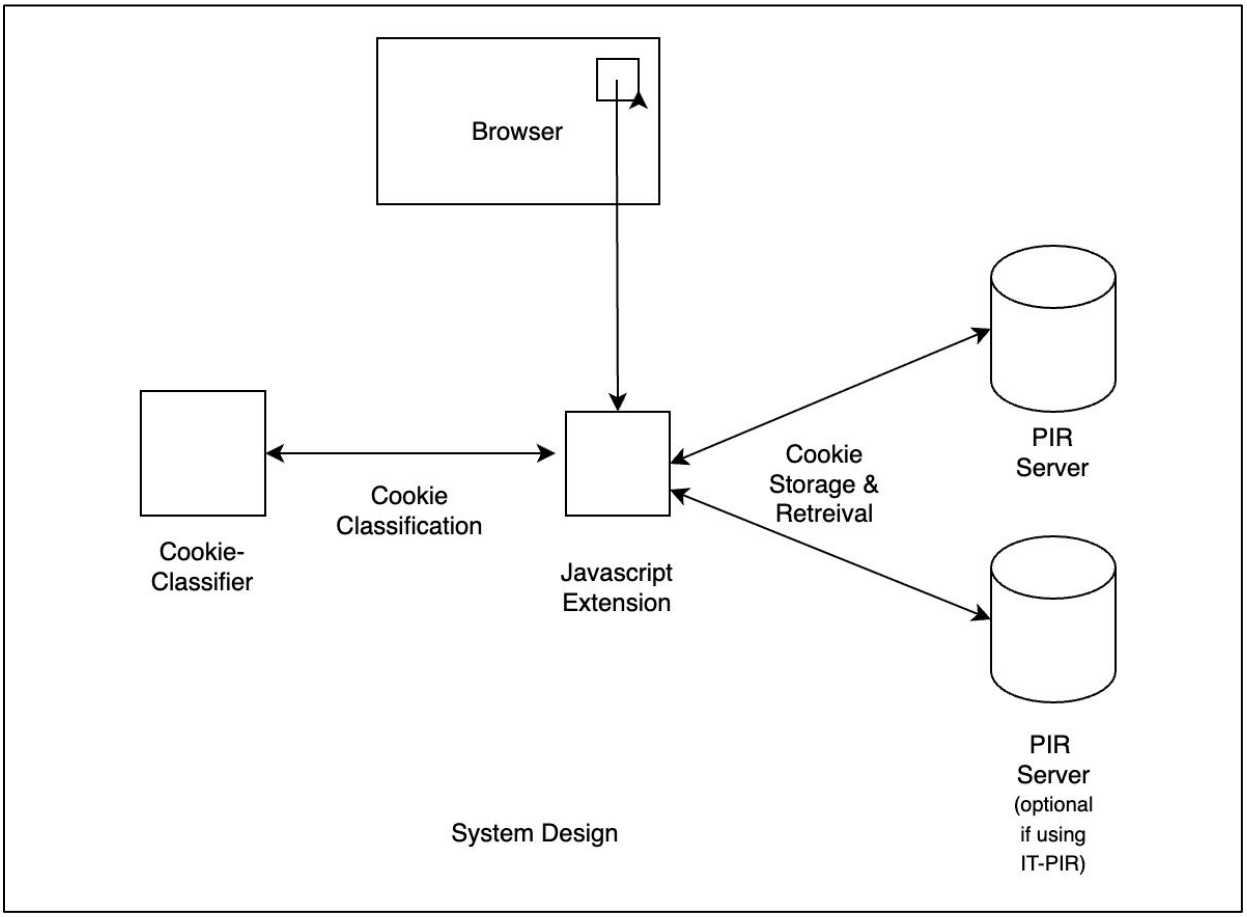

Fig 4.1 – System Design Architecture

4.1.1 Cookie Monitoring and Classification

A browser extension acts as the client agent, observing cookies as they are set or accessed. It captures attributes (HttpOnly, Secure, SameSite), contextual metadata (domain, third/first-party, expiry, script source), and lexical features (name tokens, entropy). These features are processed by a hybrid ML pipeline: a lightweight lexical model for fast filtering and a contextual/behavioral model for complex cases. The classifier outputs a cookie category (essential, functional, analytics, marketing) along with a sensitivity score, enabling selective handling.

4.1.2 Privacy-Aware Storage and Retrieval

Based on classification, non-sensitive cookies are cached locally to preserve performance, while sensitive cookies are encrypted and outsourced to PIR-enabled storage. Homomorphic encryption or optimized protocols like PIR ensure secure retrieval without revealing access patterns. A policy engine governs which cookies are outsourced, minimizing overhead. A simple dashboard provides users with visibility, override options, and short explanations for classification outcomes.

4.2 Design

4.2.1 Data Flow Diagram

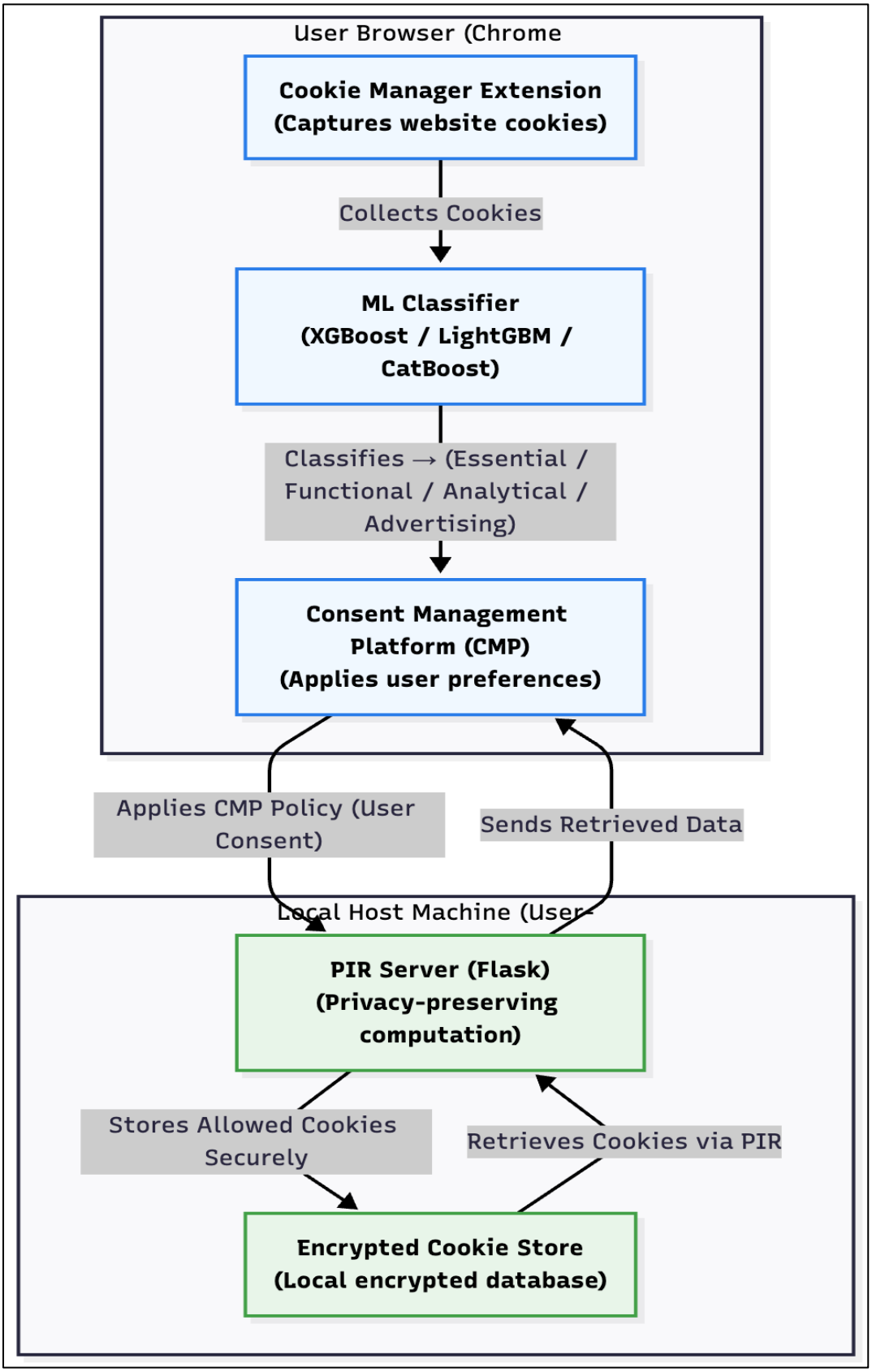

Fig 4.2 – Data Flow Diagram

4.2.2 Use Case Diagram

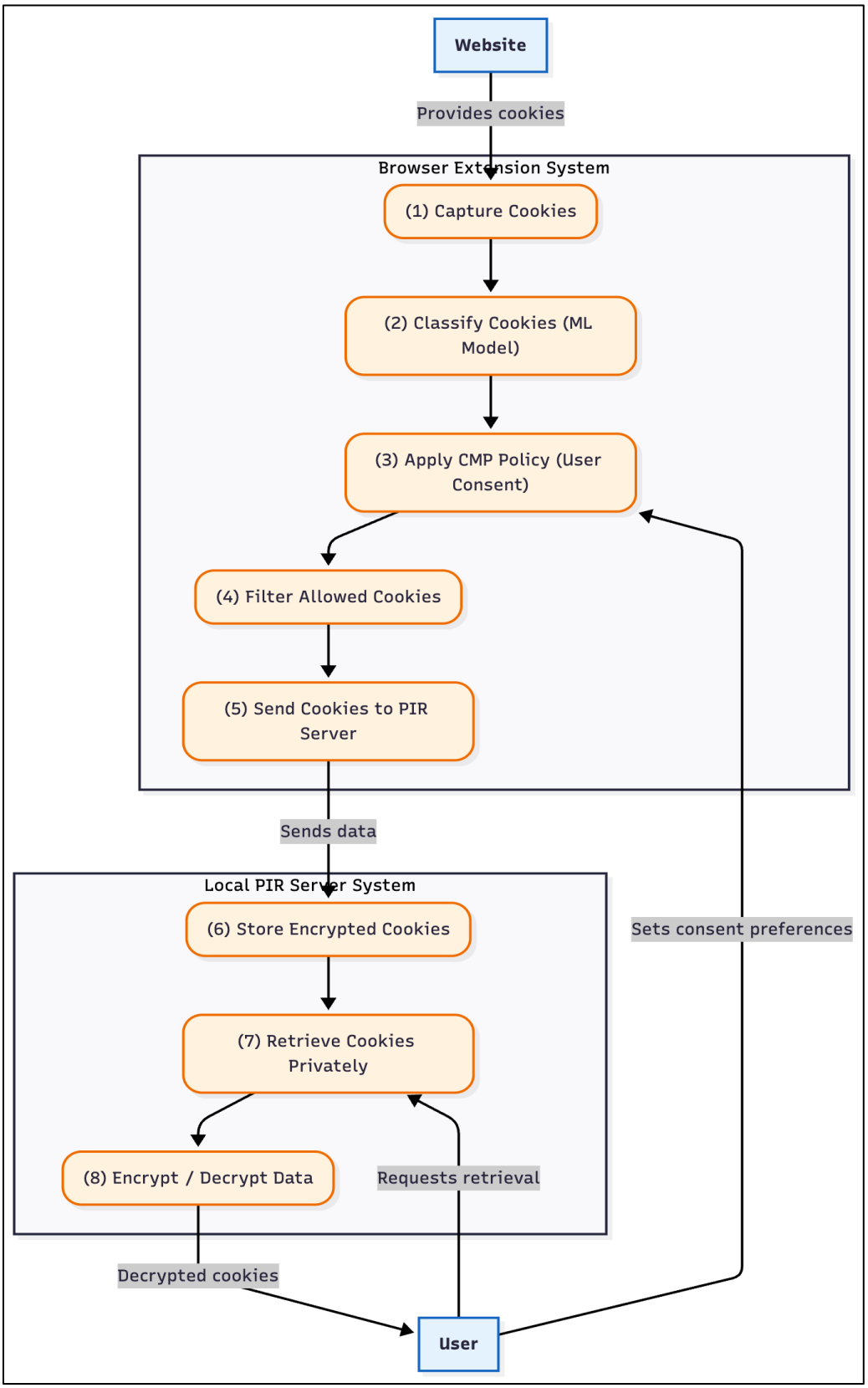

Fig 4.3 – Use Case Diagram

4.2.3 Class Diagram

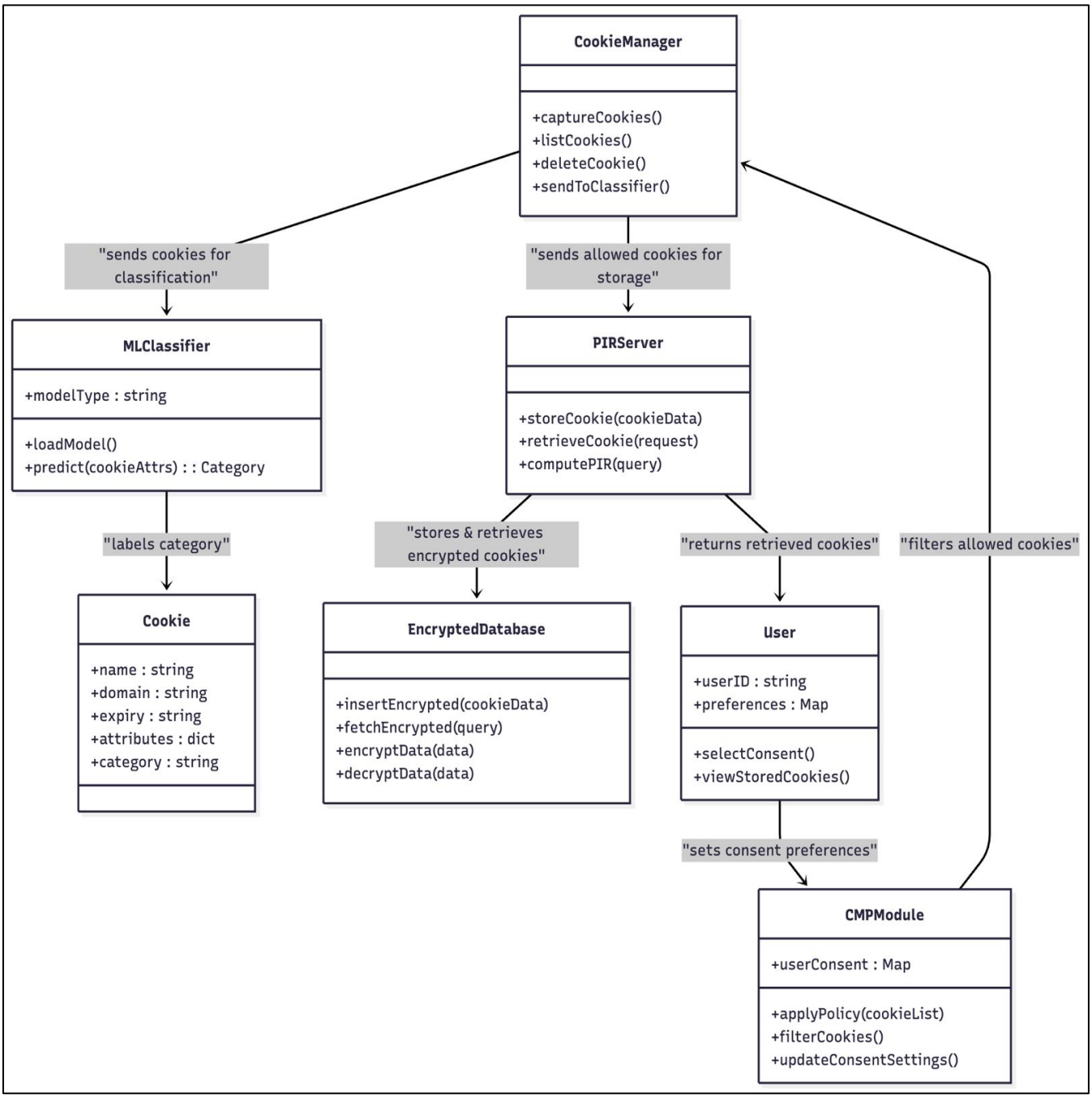

Fig 4.4 – Class Diagram

4.2.4 Sequence Diagram

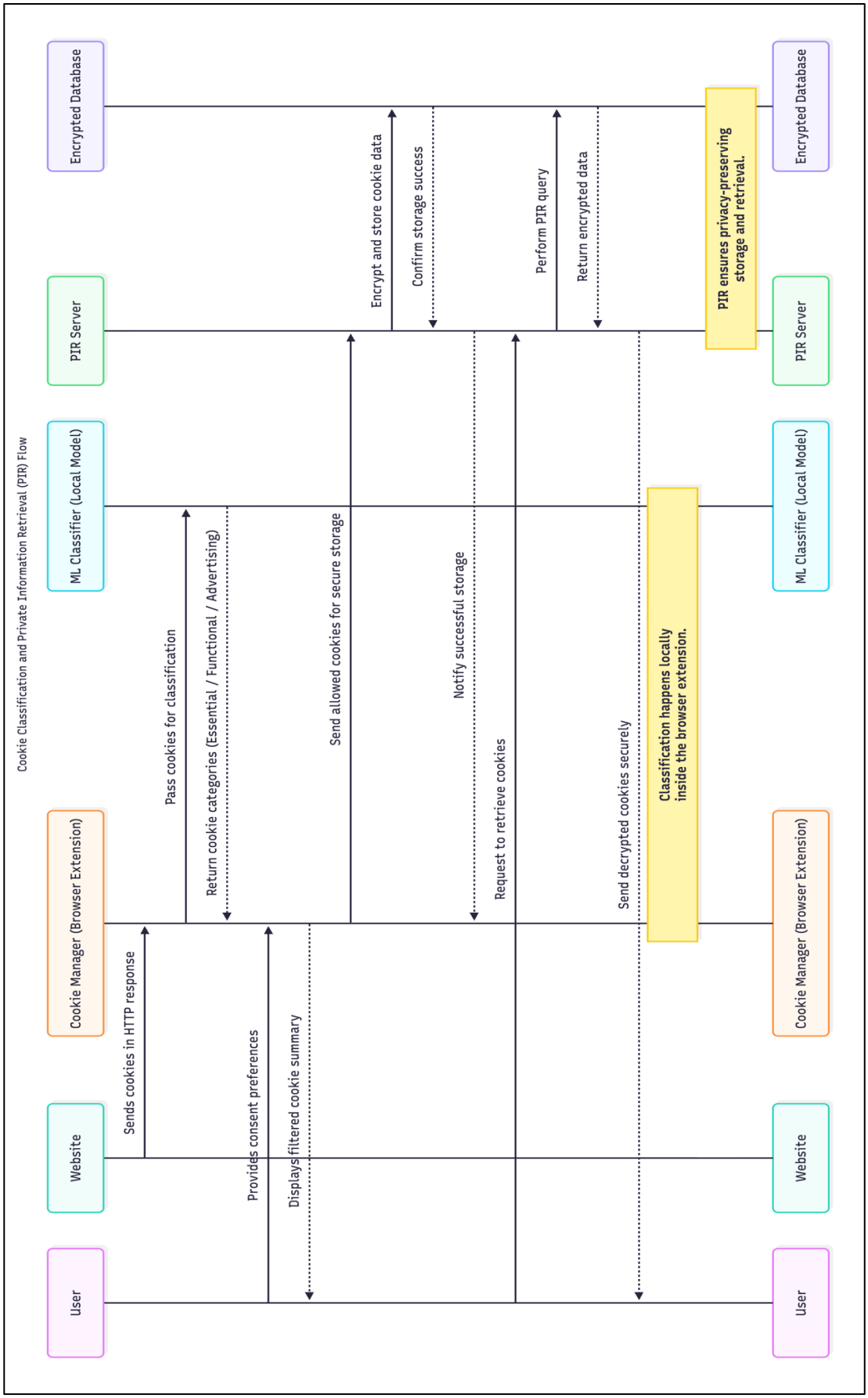

Fig 4.5 – Sequence Diagram

CHAPTER 5

5. METHODOLOGY AND TESTING

5.1 Methodology

5.1.1. Data Collection and Preprocessing

- Cookies were scraped from a set of targeted domains using browser automation tools and platforms such as OpenWPM, with parallel capture of category labels from active Consent Management Platforms (CMPs).
- The attributes extracted from each cookie include: name, domain, Secure flag, HttpOnly flag, SameSite flag, expiry timestamp, third-party status, and contextual metadata.
- Preprocessing involved deduplication, normalization of domains, transformation of boolean values to binary encoding, and conversion of categorical features (e.g., SameSite, category labels) into a format suitable for machine learning algorithms.

5.1.2. Machine Learning Model Development

- Supervised classification models including XGBoost, LightGBM, and CatBoost were selected.
- The labelled dataset was divided into training and test partitions; models were trained to categorize cookies into essential, functional, analytics, and marketing classes.
- Model selection was based on standard metrics such as accuracy, precision, recall, and F1-score.
- The chosen best-performing classifier was exported in ONNX format for lightweight browser inference.

5.1.3. Private Information Retrieval (PIR) Integration

- Sensitive cookies, as identified by the ML model, are outsourced to secure storage using PIR protocols.
- PIR or comparable homomorphic encryption-based schemes were implemented for communication-efficient and high-rate retrieval, preserving the privacy of access patterns.
- The PIR subsystem was integrated via a modular backend service responsible for encrypted storage and retrieval, ensuring compliance with privacy goals.

5.1.4. System Demonstration

- The core system workflow initiates with the installation of a browser extension, developed by our team, that integrates the exported ML classifier to run real-time inference within the browser environment.
- The extension listens for cookie events, extracts relevant features, and uses the embedded ML model to classify cookies instantly.
- Users can specify site-level cookie storage preferences via the extension interface, which then automates the removal of non-essential cookies or outsources sensitive cookies following privacy policies.

- All relevant activity, including classification decisions and cookie management actions, is logged for auditing and compliance monitoring.
- This integration enables a seamless, privacy-preserving cookie management system that enforces user preferences while leveraging state-of-the-art ML and PIR technologies.

5.2 Testing

5.2.1 Unit Testing

Each module was tested independently to verify correctness and robustness:

- **Extension Core**: Validated event listeners, cookie parsing, feature vector formation, and correct invocation of the classifier.
- **Machine Learning**: Assured accurate inference, correct boundary conditions, and robust error handling for edge cases (such as malformed cookies).
- **PIR Module**: Verified correctness of encryption/decryption routines, request/response integrity, and proper functioning of the privacy logic.

5.2.2 Integration Testing

Interaction between modules was tested using integration scenarios:

- **Browser Extension & Classifier**: Confirmed that detected cookies are correctly classified and labels are used to trigger downstream actions.
- **Classifier & PIR Backend**: Ensured seamless handover of sensitive cookie data from the extension to the PIR server, and correct retrieval on-demand.
- **End-to-End Data Flow**: Simulated typical browser usage (site visits, user settings changes, cookie events), validating that classification, removal, and outsourcing operate as intended in realistic scenarios.

5.2.3 System and Acceptance Testing

- **System Testing**: The full system (extension + backend) was evaluated for usability, performance (latency, overhead), and functional accuracy, using test cases based on actual website visits and cookie activity.
- **Security and Compliance**: Thorough validation ensured that GDPR/CCPA requirements were met, no leakage of sensitive cookie data or user patterns occurred.
- **User Experience**: The dashboard and UI were tested for responsiveness, real-time updates, and clarity in presenting classification results and privacy actions.
- **Logging and Reporting**: Every significant operation was logged by the extension (and the backend service worker), enabling traceability and compliance auditing.

CHAPTER 6

6. PROJECT DEMONSTRATION

This chapter demonstrates the complete working of the PIR Cookie Manager browser extension and its integration with the backend PIR Server for privacy-preserving cookie classification and management. The workflow covers initialization, cookie event listening, feature extraction, model inference, preference-based removal, logging, and dashboard monitoring.

Workflow

Step 1 - Initialization:

When the extension is installed or started, the background script (background.js) loads the trained model (cookie_classifier.onnx) using libs/ort.min.js through a WebAssembly backend. Optionally, the same classification can be handled on a Flask server using a Joblib/CBM model.

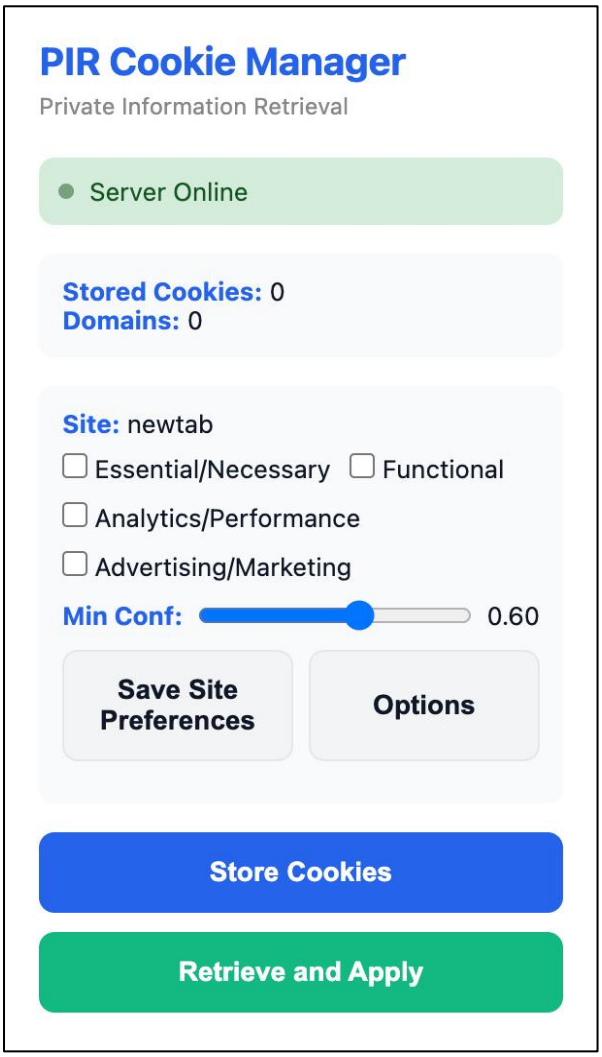

Fig 6.1 – PIR Cookie Manager interface showing model initialization, server status, and configurable site-level cookie preferences.

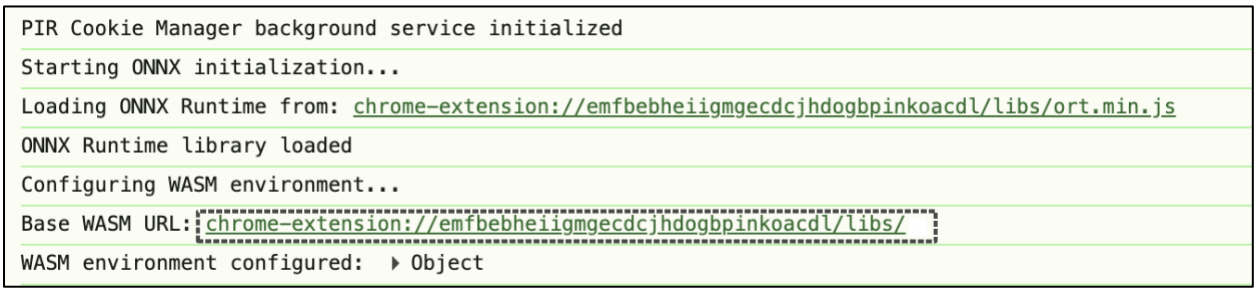

Fig 6.2 – Service worker logs showing model initialization and server connection

Step 2 - Event Listening:

The extension continuously monitors browser activity, listening for cookie creation, modification, and deletion events. It also intercepts `Set-Cookie` headers as the user browses.

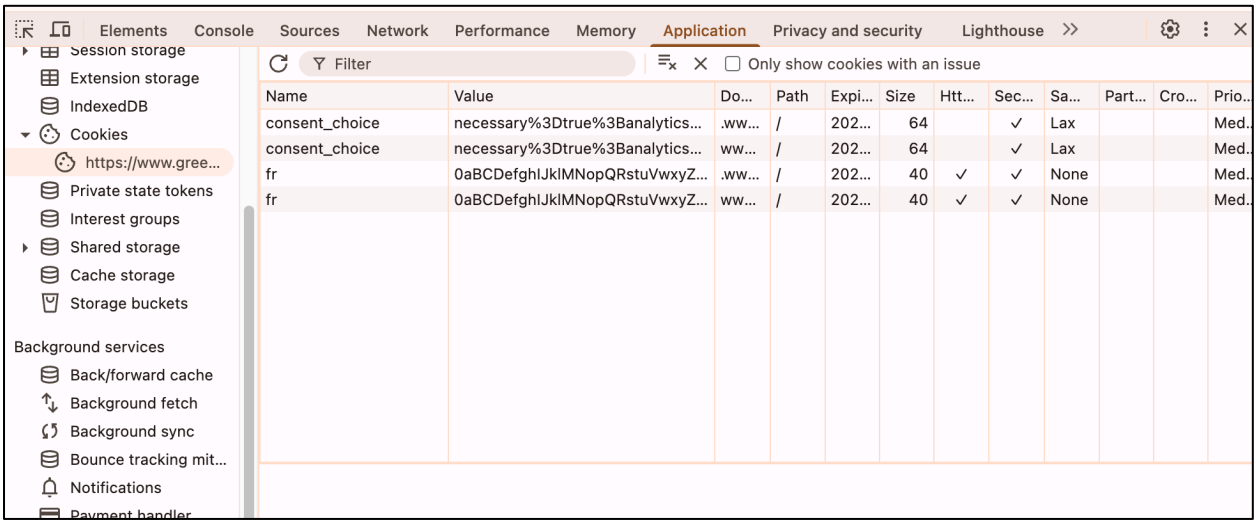

Fig 6.3 – Application Dev Tool showing current cookies being stored in the browser

Cookies being set trigger the event listener which in turn runs the worker for classifying the cookie.

Step 3 - Feature Extraction:

For every cookie, it constructs a feature vector containing attributes such as name, domain, path, size, Secure/HttpOnly/SameSite flags, expiry, and third-party status. The model also considers other features like name prefixes and one-hot labels them to serve as an input to the model.

Features such as has_ga , has_id are derived from the cookie name which are one-hot-encoded into 0/1 based on the Boolean value .

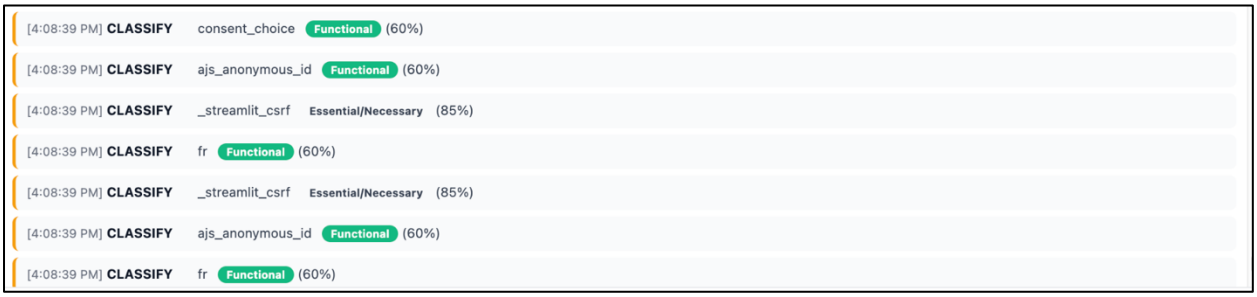

Figure 6.4 – PIR Cookie Extension dashboard showing cookie classification

Step 4 - Model Inference:
The ONNX model predicts the cookie's category and confidence score locally. For server based classification the native Joblib/CBM model is used.

The model classifies the cookie into one of the 4 categories (Essential, Functional, Analytical, Marketing) along with a confidence score. Based on the minimum confidence score the cookie is then passed on for further action.

If it falls under unallowed category set by the user the cookie(s) are removed from the browser by the extension

```
Removed blocked cookie after sync: OTZ @ ogs.google.com
Removed blocked cookie after sync: __clerk_db_jwt @ greenfirst-storage.rayiq.ai
Auto-sync complete: stored=0 blocked=0
Removed blocked cookie after sync: OTZ @ ogs.google.com
Removed blocked cookie after sync: __cf_bm @ tender-hermit-73.clerk.accounts.dev
Removed blocked cookie after sync: __clerk_db_jwt_hgVorJD3 @ greenfirst-storage.rayiq.ai
Removed blocked cookie after sync: _cfuvid @ tender-hermit-73.clerk.accounts.dev
Removed blocked cookie after sync: NID @ google.com
Removed blocked cookie after sync: __client_uat @ rayiq.ai
Removed blocked cookie after sync: __clerk_db_jwt @ greenfirst-storage.rayiq.ai
Removed blocked cookie after sync: __clerk_db_jwt_hgVorJD3 @ greenfirst-storage.rayiq.ai
Removed blocked cookie after sync: __client_uat_hgVorJD3 @ rayiq.ai
Auto-sync started...
Auto-sync complete: stored=0 blocked=0
```

Fig 6.5 – Service worker logs showing cookie removal

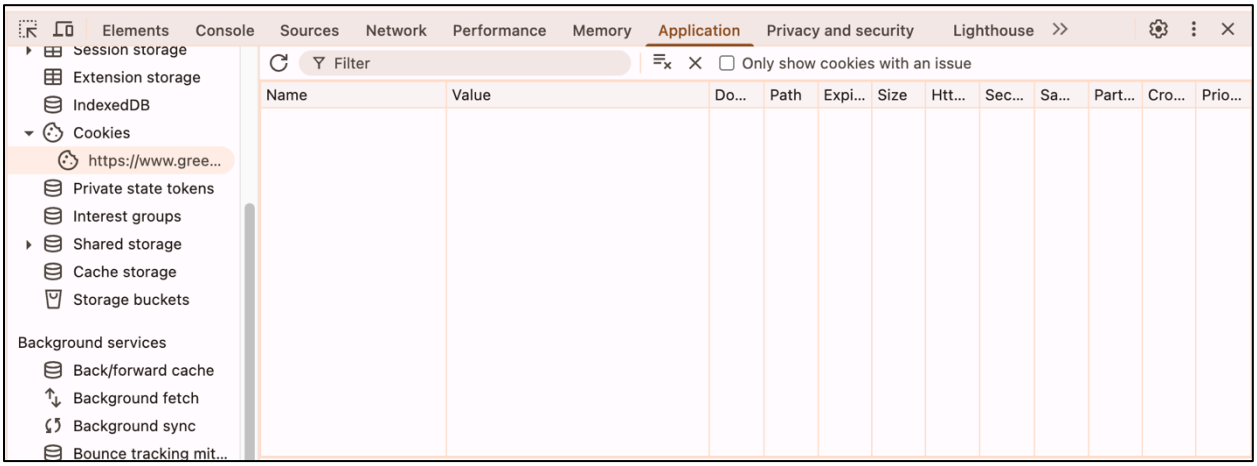

Figure 6.6 – Application Dev tools can be checked to see if the unwanted cookies still exist.

Step 5 - Result Storage & Display:
Predictions are stored (e.g., via chrome.storage) and reflected in:

1. **popup.html / popup.js:** shows recent cookies with classifications.
2. **dashboard.html / dashboard.js:** provides a full table with filters, search, and detailed insights.

Complete monitoring of the cookie handling can be seen on the dashboard.html file.

You can save global preferences (to enable as default on any website you visit) and even setup a global confidence level.

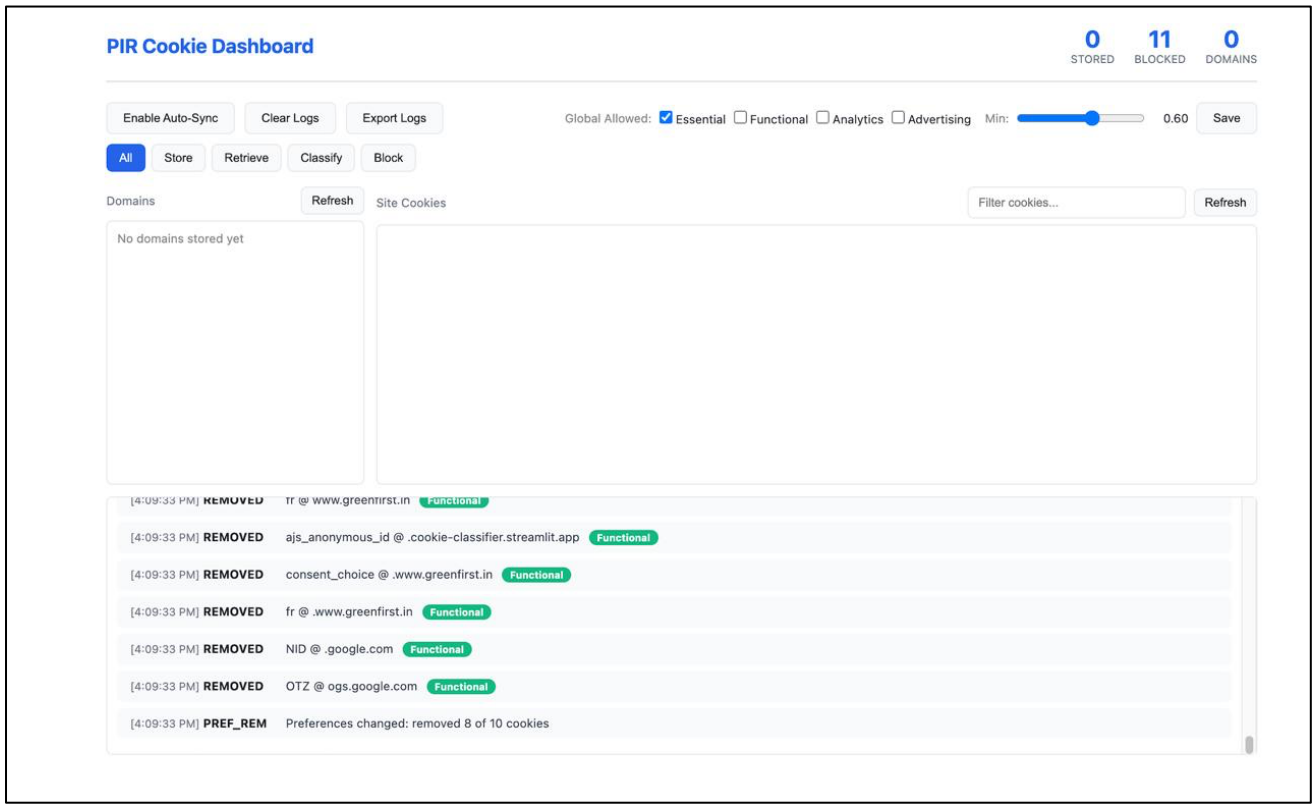

Figure 6.7 – PIR cookie dashboard showing cookie handling

End to End Operations :

When users visit a new site, they can set cookie preferences and save them. The extension enforces these rules automatically-classifying, storing, or deleting cookies in accordance with user choices. All background activity, such as model loading, synchronization, and blocked cookie removal, is logged for audit and transparency.

CHAPTER 7

7. RESULT AND DISCUSSION

7.1 Overview

This chapter presents the experimental results and analysis of the cookie classification framework. The goal of the project was to accurately classify cookies into four categories: Essential, Functional, Analytics, and Marketing. It also explores how the system can use Private Information Retrieval (PIR) to securely manage non-essential cookies while preserving user privacy. Three supervised ensemble models were trained and evaluated: XGBoost, LightGBM, and CatBoost. Each model was trained on 70% of the dataset, validated on 15%, and tested on the remaining 15%. Class stratification was maintained throughout all splits to ensure balanced representation.

7.2 Model Performance Evaluation

The bar chart in Figure 7.1 illustrates the comparative accuracy of the three ensemble-based models - LightGBM, XGBoost, and CatBoost used for cookie classification. It can be observed that LightGBM achieved the highest accuracy, followed closely by XGBoost, while CatBoost showed slightly lower performance. This visual comparison highlights LightGBM's superior ability to generalize across different cookie types, making it the most effective model for this task.

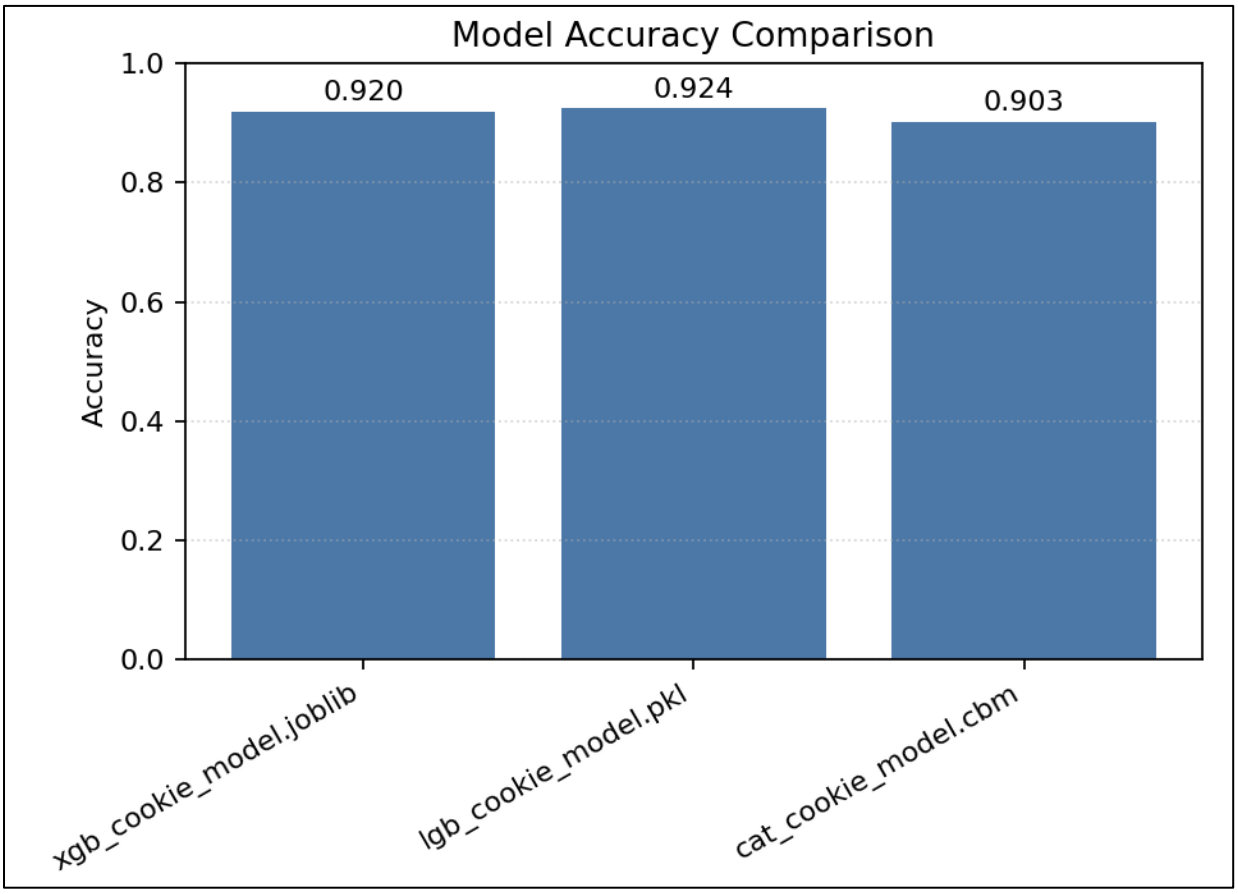

Figure 7.1: Showing the accuracy comparison between the models

The detailed numerical results are summarized in Table 7.1, where the models are evaluated using four metrics: Accuracy**,** Precision (Macro)**,** Recall (Macro), and F1-Score (Macro)

Table 7.1: Model Performance Comparison

Model	**Accuracy**	**Precision (Macro)**	**Recall (Macro)**	**F1-Score (Macro)**
LightGBM	0.925	0.901	0.878	0.888
XGBoost	0.920	0.899	0.87	0.884
CatBoost	0.903	0.879	0.848	0.861

LightGBM achieved the highest accuracy of 92.45% and performed better across all metrics compared to XGBoost and CatBoost. The improvement in precision and F1-score shows that the model could accurately detect minority classes such as marketing and analytics cookies while keeping false positives low.

7.3 Confusion Matrix Analysis

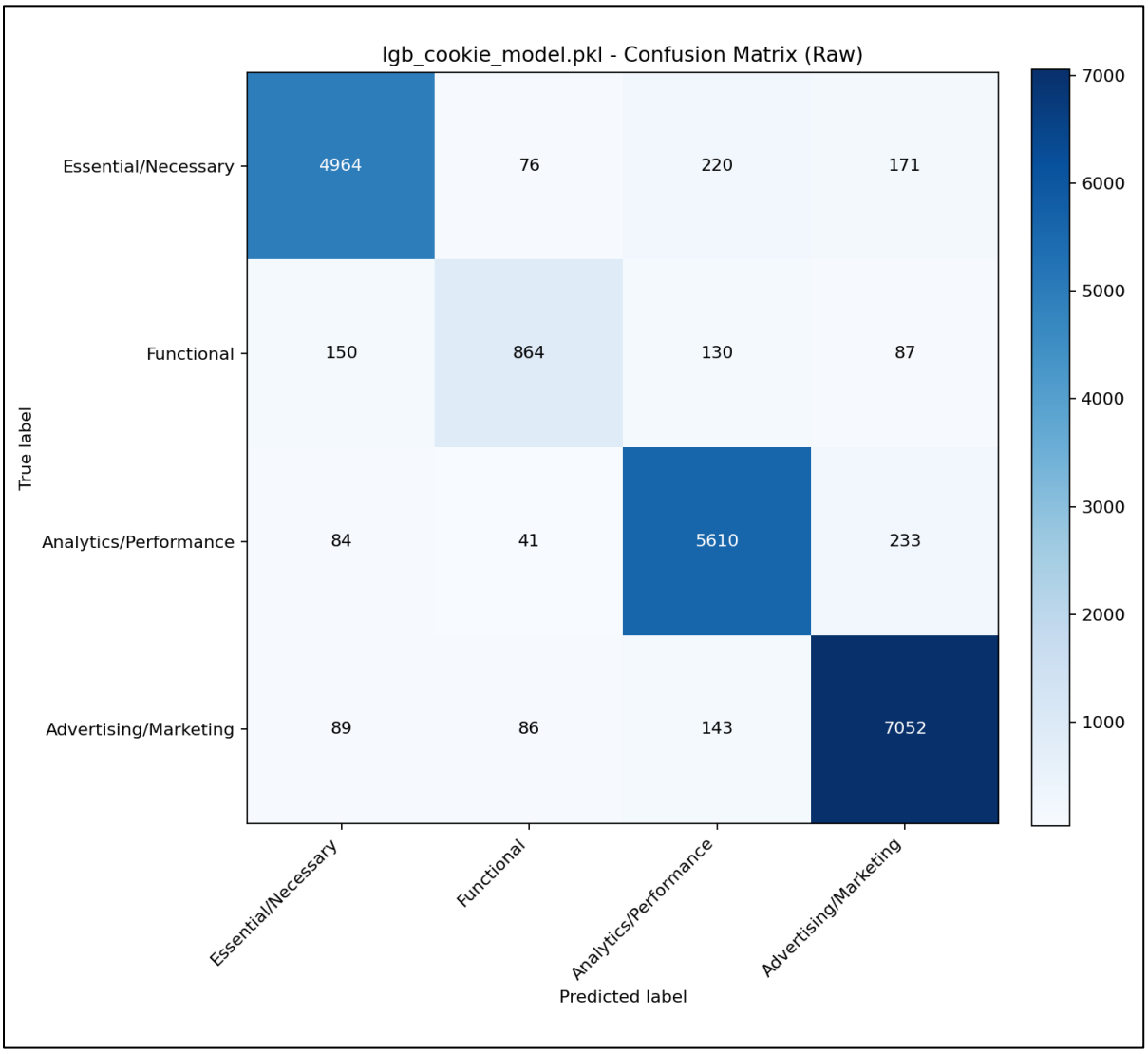

Fig 7.2 Confusion matrix showing the classification distribution across cookie categories.

To understand how the model performs across different cookie types, the confusion matrix of the LightGBM model was analyzed. The matrix is shown in Figure 7.2.

Interpretation:

- Most Most of the misclassifications occurred between functional and analytics cookies because these categories often share similar attributes such as SameSite and Secure flags.
- Essential cookies were identified with high precision (above 93 percent), confirming that the model can reliably recognize necessary cookies used for authentication and sessions.
- Marketing cookies achieved a recall greater than 94 percent, showing that the model performs well in identifying cookies related to advertising and third-party tracking.

Table 7.2 summarizes the category-wise performance of the LightGBM model.

Cookie Type	Precision	Recall	F1-Score	Support
Essential	0.939	0.914	0.926	5431
Functional	0.810	0.702	0.752	1231
Analytics	0.919	0.940	0.930	5968
Marketing	0.935	0.957	0.946	7370

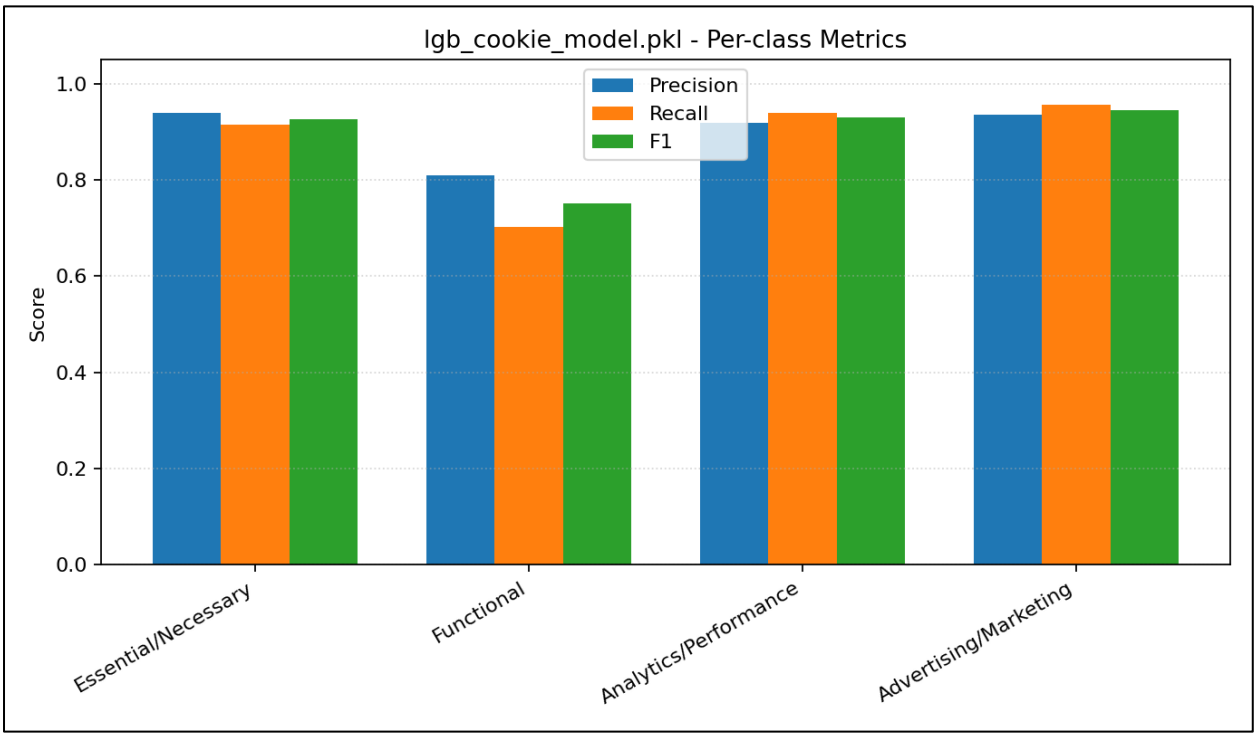

Fig 7.3 Category-wise F1-score distribution for the LightGBM model.

7.4 Integration with Private Information Retrieval

After the cookies were classified into four categories, the system connected with a Computational Private Information Retrieval (CPIR) module to securely manage and store non-essential cookies.

In this setup, the browser extension acts as the client component that communicates with the backend server. The process works in the following steps:

1. The extension receives a secure key generated by the server.
2. Cookie metadata such as domain, expiry, and category is encrypted using this key before being sent to the server.
3. The server processes the encrypted data, performs the required operations, and returns an encrypted response.
4. The extension decrypts the response locally, ensuring that all cookie data remains private and secure during transmission.

This setup functions like a lightweight PIR system where the server can process requests without accessing or identifying the actual cookie data.

7.5 Discussion

The integration of machine learning and CPIR creates a practical and privacy-focused approach for cookie management in modern browsers.

Key Findings

1. **Model Performance:** LightGBM achieved the best performance with an accuracy of 92.45 percent and an F1-score of 0.888. It was efficient and lightweight, making it suitable for real-time client-side classification.
2. **Privacy with CPIR:** The CPIR mechanism ensures that all cookie metadata transmitted to the server remains encrypted. The server operates only on encrypted data and cannot view or infer any sensitive information.
3. **Selective Outsourcing:** Only non-essential cookies are encrypted and sent for analysis or aggregation. Essential cookies stay on the device, maintaining website functionality and user sessions.
4. **System Integration:** The browser extension acts as the main control center. It classifies cookies locally, encrypts non-essential data, and securely communicates with the backend server using the CPIR mechanism.

7.6 Limitations and Future Scope

- The recall for functional cookies remains slightly lower at around 70 percent, which indicates that adding more contextual features such as network activity or request origin could help improve accuracy.
- Currently, cookies are stored in the browser for a short moment before being removed by the extension. For this system to function as a complete middleware layer, direct browser-level integration would be necessary.
- The current CPIR implementation uses a simplified encryption approach. A more advanced setup using homomorphic encryption or multi-server PIR could offer stronger privacy guarantees but would require higher computational resources.

CHAPTER 8

8. CONCLUSION

This project presents a complete framework for automatic cookie classification using supervised machine learning models, designed to improve transparency and privacy in web data management. By analyzing key cookie attributes such as domain, expiry, flags, and contextual metadata, the system classifies cookies into four main categories: Essential, Functional, Analytics, and Marketing. This classification helps users and developers better understand how websites track user activity and supports the development of smarter cookie management tools.

Among the models tested, LightGBM achieved the best performance, with an accuracy of 92.45% and a macro F1-score of 0.888, outperforming both XGBoost and CatBoost. The model showed strong generalization across all cookie types, especially in identifying essential and marketing cookies, which tend to have more distinct attribute patterns. These results confirm that gradient boosting models are well-suited for structured metadata classification and can handle class imbalances effectively in real-world datasets.

The project also demonstrates the advantages of running inference directly on the user's device using ONNX Runtime Web**.** This allows the trained model to operate within a browser extension without relying on external servers, ensuring faster processing, improved privacy, and greater user control.

In addition to classification, the system includes a simple client-side encryption mechanism for secure handling of selected non-essential cookies. This feature, inspired by Private Information Retrieval (PIR) concepts, ensures that cookie-related information can be shared or stored safely without exposing sensitive details.

In conclusion, this project provides a practical and deployable approach to cookie classification and privacy management. It successfully combines machine learning accuracy with user-side privacy protection and transparency. Future work can build on this foundation by incorporating deeper contextual analysis, such as consent banner behavior or network request data, and by expanding the dataset for broader coverage. Such developments can further strengthen user privacy and contribute to a safer and more transparent web environment.

CHAPTER 9

9. REFERENCES

1. Berens, B. M., Bohlender, M., Dietmann, H., Krisam, C., Kulyk, O., & Volkamer, M. (2024). Cookie disclaimers: Dark patterns and lack of transparency. *Computers & Security, 135,* 103507.

2. Kosztyán, Z. T., Király, T., Csizmadia, T., Katona, A. I., & Vathy-Fogarassy, Á. (2025). Automated research methodology classification using machine learning. *Engineering Applications of Artificial Intelligence, 132,* 107007.

3. Hu, X., Sastry, N., & Mondal, M. (2021). Corralling cookies into categories with CookieMonster. In *Proceedings of the ACM Web Science Conference (WebSci '21)* (pp. 56–65). ACM.

4. Munir, S., Siby, S., Iqbal, U., Englehardt, S., Shafi, Z., & Troncoso, C. (2023). Understanding and detecting first-party tracking cookies. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)* (pp. 2345–2360). ACM.

5. Bouhoula, A., Kubicek, K., Zac, A., Cotrini, C., & Basin, D. (2023). Automated large-scale analysis of cookie notice compliance. In *32nd USENIX Security Symposium (USENIX Security 23)* (pp. 4069–4086). USENIX Association.

6. Bollinger, D., Kubicek, K., Cotrini, C., & Basin, D. (2022, August). Automating cookie consent and GDPR violation detection. In *31st USENIX Security Symposium (USENIX Security 22)* (pp. 2893–2910). USENIX Association.

7. Sivan-Sevilla, I., Parham, P., & McGuigan, L. (2025). 'Cookie-less' identification for/against privacy? *Internet Policy Review, 14*(3).

8. Vithana, S. V., Wang, Z., & Ulukus, S. (2023). Private information retrieval and its applications. *arXiv.*

9. Luo, M., & Wang, M. (2025). Faster Spiral: Low-communication, high-rate private information retrieval. *Cryptography, 9*(1), 13.

10. Koçyiğit, E., Rossi, A. and Lenzini, G., 2022. Towards assessing features of dark patterns in cookie consent processes. *Proceedings of the IFIP International Summer School on Privacy and Identity Management*, Springer.

11. Gray, C., 2021. Dark patterns and the legal requirements of consent: framing the problem and the way forward. *SSRN Electronic Journal.*

12. Mejtoft, T., Frängsmyr, E., Söderström, U. and Norberg, O., 2021. Deceptive design: cookie consent and manipulative patterns. *Proceedings of BLED 2021 Conference.*

13. Hausner, P. and Gertz, M., 2021. Dark patterns in the interaction with cookie banners. *CHI Workshop on Dark Patterns in HCI.*

14. Pantelić, O., Jović, K. and Krstović, S., 2022. Cookies implementation analysis and the impact on user privacy regarding GDPR and CCPA regulations. *Sustainability*, 14(9), p.5015.

15. Luo, M. and Wang, M., 2024. Faster FHE-based single-server private information retrieval. *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*.

16. Heidarzadeh, A. and Sprintson, A., 2022. The role of reusable and single-use side information in private information retrieval (PIR-RSSI). *arXiv preprint arXiv:2201.11605.*

17. Gomez-Leos, A. and Heidarzadeh, A., 2022. Single-server private information retrieval with side information under arbitrary popularity profiles. *arXiv preprint arXiv:2205.06172.*

18. Mozaffari, H. and Houmansadr, A., 2023. Heterogeneous private information retrieval (HPIR). *Network and Distributed System Security (NDSS) Symposium Proceedings.*

19. Wang, Z. and Ulukus, S., 2022. Symmetric private information retrieval at the private information retrieval rate. *IEEE Journal on Selected Areas in Communications*, 40(6).

20. Arora, S., Lewis, P., Fan, A., Kahn, J. and Ré, C., 2022. Reasoning over public and private data in retrieval-based systems. *arXiv preprint arXiv:2203.11027.*

21. Li, X., Wang, G., Liang, P., Yang, S., Li, G. and Liu, Y., 2025. Efficient private information retrieval scheme with dynamic database – MKFHE-based. *Electronics*, 14(17), p.3441.

APPENDIX A – Sample Code

Dataset collection

reachability_crawler

```
def run_reachability_check(self, input_domain: str) -> Tuple[Optional[str], int]:
        component_tuple = urlparse(input_domain)
        if component_tuple.scheme in ("http", "https"):
            url_suffix = input_domain
            prefix_list = [""]
        else:
            url_suffix = re.sub(r"^www\.", "", input_domain)
            prefix_list = ["https://www.", "https://", "http://"]

        final_url = None
        r = None

        # Try different URL prefixes
        for prefix in prefix_list:
            completed_url = prefix + url_suffix

            try:
                headers = {
                    'User-Agent': "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/70.0.3538.77 Safari/537.36"
                }
                r = requests.get(completed_url, timeout=(connect_timeout,
load_timeout), headers=headers)
            except (rexcepts.TooManyRedirects, rexcepts.SSLError,
                    rexcepts.URLRequired, rexcepts.MissingSchema):
                if debug_mode:
                    logger.debug(f"SSL/Schema error for: '{completed_url}'")
                return input_domain, QuickCrawlResult.CONNECT_FAIL
            except (rexcepts.ConnectionError, rexcepts.Timeout):
                if debug_mode:
                    logger.debug(f"Connection/timeout error for: '{completed_url}'")
                continue
            except Exception as ex:
                if debug_mode:
                    logger.error(f"Unexpected error for '{completed_url}': {ex}")
                return input_domain, QuickCrawlResult.CONNECT_FAIL

            if r is None:
                continue
            elif not r.ok:
                # Bot detection responses
                if r.status_code in (403, 406):
                    return completed_url, QuickCrawlResult.BOT
                else:
                    return completed_url, QuickCrawlResult.HTTP_ERROR
            else:
                final_url = r.url
                break
```

```
        # Check for CMP presence if we got a successful response
        if final_url is not None and check_cmp and r is not None:
            # Check each CMP type
            if self.check_cookiebot_presence(r):
                return final_url, QuickCrawlResult.COOKIEBOT
            elif self.check_onetrust_presence(r):
                return final_url, QuickCrawlResult.ONETRUST
            elif self.check_termly_presence(r):
                return final_url, QuickCrawlResult.TERMLY
            else:
                return final_url, QuickCrawlResult.NOCMP
        elif final_url is not None:
            return final_url, QuickCrawlResult.OK
        else:
            return input_domain, QuickCrawlResult.CONNECT_FAIL
```

crawl domain

```
def crawl_domain(self, domain: str) -> CrawlResult:
        """Crawl a single domain and collect cookie consent data"""
        logger.info(f"Crawling domain: {domain}")

        driver = None
        try:
            # Create browser driver
            driver = self.create_driver()

            # Navigate to domain
            if not domain.startswith(("http://", "https://")):
                domain = f"https://{domain}"

            driver.get(domain)

            # Wait for page to load
            WebDriverWait(driver, 15).until(
                lambda d: d.execute_script("return document.readyState") == "complete"
            )

            # Detect CMP type
            cmp_type = self.detect_cmp_type(driver)
            logger.info(f"Detected CMP: {cmp_type} for {domain}")

            # Extract consent data based on CMP type
            consent_data = []
            if cmp_type == "cookiebot":
                consent_data = self.extract_consent_data_cookiebot(driver)
            elif cmp_type == "onetrust":
                consent_data = self.extract_consent_data_onetrust(driver)

            # Collect cookies
            cookies = self.collect_cookies(driver)

            # Interact with consent banner if present (accept all)
            try:
```

```
            # Look for common accept buttons
            accept_selectors = [
                "[id*='accept']", "[class*='accept']",
                "[id*='agree']", "[class*='agree']",
                "[aria-label*='Accept']", "[aria-label*='Agree']"
            ]

            for selector in accept_selectors:
                try:
                    button = driver.find_element(By.CSS_SELECTOR, selector)
                    if button.is_displayed() and button.is_enabled():
                        button.click()
                        logger.debug(f"Clicked consent button: {selector}")
                        time.sleep(2)  # Wait for consent processing
                        break
                except:
                    continue
        except Exception as e:
            logger.debug(f"No consent banner interaction needed: {e}")

        # Collect cookies again after consent
        final_cookies = self.collect_cookies(driver)

        return CrawlResult(
            domain=domain,
            success=True,
            cmp_type=cmp_type,
            cookies_collected=len(final_cookies),
            consent_data=consent_data
        )

    except TimeoutException:
        return CrawlResult(
            domain=domain,
            success=False,
            cmp_type="unknown",
            cookies_collected=0,
            consent_data=[],
            error_message="Page load timeout"
        )
    except Exception as e:
        return CrawlResult(
            domain=domain,
            success=False,
            cmp_type="unknown",
            cookies_collected=0,
            consent_data=[],
            error_message=str(e)
        )
    finally:
        if driver:
            try:
                driver.quit()
```

```
                except:
                    pass
```

Model Training

lightgbm_train.py

```
def train_lightgbm(
    X: np.ndarray,
    y: np.ndarray,
    output_dir: str = "trained_models",
    model_name: str = "lgb_cookie_model.pkl",
) -> Dict:
    try:
        import lightgbm as lgb  # type: ignore
    except Exception as e:
        return {
            "accuracy": 0.0,
            "model_path": "N/A",
            "onnx_path": None,
            "skipped": True,
            "reason": f"lightgbm not installed: {e}",
        }
    mask = y >= 0
    X_use = X[mask]
    y_use = y[mask]
    X_train, X_val, y_train, y_val = train_test_split(
        X_use, y_use, test_size=0.2, random_state=42, stratify=y_use if
len(set(y_use)) > 1 else None
    )
    train_ds = lgb.Dataset(X_train, label=y_train)
    val_ds = lgb.Dataset(X_val, label=y_val, reference=train_ds)
    num_classes = int(np.max(y_use)) + 1 if y_use.size else 4
    params = {
        "objective": "multiclass",
        "num_class": num_classes,
        "learning_rate": 0.1,
        "num_leaves": 64,
        "feature_fraction": 0.8,
        "bagging_fraction": 0.9,
        "metric": "multi_logloss",
        "verbosity": -1,
    }

    callbacks = []
    try:
        # Newer LightGBM uses callbacks for early stopping
        callbacks.append(lgb.early_stopping(30))
    except Exception:
        pass
    model = lgb.train(params, train_ds, valid_sets=[val_ds], num_boost_round=500,
callbacks=callbacks)
    preds = model.predict(X_val)
    y_pred = np.argmax(preds, axis=1)
    acc = float(accuracy_score(y_val, y_pred)) if y_val.size else 0.0
```

```
    _ensure_dir(output_dir)
    model_path = os.path.join(output_dir, model_name)
    joblib.dump(model, model_path)
    # Optional ONNX export
    onnx_path: Optional[str] = None
    try:
        from onnxmltools import convert_lightgbm  # type: ignore
        from onnxmltools.convert.common.data_types import FloatTensorType  # type:
ignore

        onnx_model = convert_lightgbm(
            model,
            initial_types=[("input", FloatTensorType([None, X.shape[1]]))],
        )
        onnx_path = os.path.join(output_dir, model_name.replace(".pkl", ".onnx"))
        with open(onnx_path, "wb") as f:
            f.write(onnx_model.SerializeToString())
    except Exception:
        onnx_path = None

    return {"accuracy": acc, "model_path": model_path, "onnx_path": onnx_path}
```

classify_cookie.py

```
def classify_cookie(self, cookie: Dict) -> Tuple[str, float, int]:
        """
        Returns (category_name, confidence, class_index)
        If model missing or failed, falls back to simple rules.
        """
        # Fallback if model missing
        if not self.ensure_loaded():
            return self._rule_based(cookie)

        try:
            import xgboost as xgb

            X = self._extract_features(cookie)
            dtest = xgb.DMatrix(X)
            y_proba = self.model.predict(dtest)
            # If booster returns shape (n_classes,) for single row
            if y_proba.ndim == 1:
                y_proba = y_proba.reshape(1, -1)
            probs = y_proba[0]
            cls_idx = int(np.argmax(probs))
            confidence = float(np.max(probs))
            category = self.label_map[cls_idx] if 0 <= cls_idx < len(self.label_map)
else str(cls_idx)
            logging.info(
                f"Classified '{cookie.get('name','')}' as '{category}'
(confidence={confidence:.2f})"
            )
            return category, confidence, cls_idx
```

```
        except Exception as exc:
            logging.error(f"XGBoost inference error: {exc}")
            return self._rule_based(cookie)
```

extract_features.py

```
def _extract_features(self, cookie: Dict) -> np.ndarray:
        """
        Compact numeric feature vector derived from cookie dict.
        Adjust this to mirror your training pipeline as needed.
        """
        name = (cookie.get("name") or "").lower()
        value = (cookie.get("value") or "")
        domain = (cookie.get("domain") or "").lower()
        path = (cookie.get("path") or "/")
        secure = self._bool(cookie.get("secure", False))
        http_only = self._bool(cookie.get("httpOnly", False))
        same_site = (cookie.get("sameSite") or "None").capitalize()
        expiration = cookie.get("expirationDate")

        # Name-based indicators
        name_len = len(name)
        has_session = 1 if ("session" in name or name in {"phpsessid", "sessionid",
"sid"}) else 0
        has_id = 1 if ("id" in name) else 0
        has_token = 1 if ("token" in name or name.endswith("_token")) else 0
        has_auth = 1 if ("auth" in name) else 0
        has_track = 1 if ("track" in name or name.startswith("trk") or
name.startswith("_gid")) else 0
        has_ga = 1 if (name.startswith("_ga") or name == "ga") else 0
        has_ad = 1 if ("ad" in name or name.startswith("_fbp")) else 0
        has_analytics = 1 if ("analytics" in name or name.startswith("_gat")) else 0

        # Value-based indicators
        value_len = len(value)
        value_is_alnum = 1 if value.isalnum() else 0

        # Domain/path
        domain_len = len(domain)
        domain_has_dot_prefix = 1 if domain.startswith(".") else 0
        path_len = len(path)

        # Security flags
        is_secure = secure
        is_http_only = http_only

        # SameSite one-hot
        is_strict = 1 if same_site == "Strict" else 0
        is_lax = 1 if same_site == "Lax" else 0
        is_none = 1 if same_site == "None" else 0

        # Expiration features
        has_expiration = 1 if expiration else 0
        expiration_ts = float(expiration) if expiration else 0.0
```

```
        features = np.array([
            name_len,
            has_session,
            has_id,
            has_token,
            has_auth,
            has_track,
            has_ga,
            has_ad,
            has_analytics,
            value_len,
            value_is_alnum,
            domain_len,
            domain_has_dot_prefix,
            path_len,
            is_secure,
            is_http_only,
            is_strict,
            is_lax,
            is_none,
            has_expiration,
            expiration_ts,
        ], dtype=float).reshape(1, -1)

        return features
```

Server

server_fetch.py

```
@app.route('/api/cookies/retrieve', methods=['POST'])
def retrieve_cookies():
    """
    Retrieve cookies using PIR query
    Can filter by domain or retrieve all
    """
    try:
        data = request.json
        domain = data.get('domain', None)
        use_pir = data.get('use_pir', False)

        if use_pir and 'encrypted_query' in data:
            # PIR query - retrieve by encrypted index
            encrypted_query = data['encrypted_query']
            all_cookies = cookie_store.get_all_cookies()

            # Process PIR query
            result = pir_system.process_pir_query(encrypted_query, all_cookies)
            try:
                cookie_logger.log_event('PIR_QUERY', {
                    'encrypted_query_prefix': encrypted_query[:16],
                    'db_size': len(all_cookies),
                    'success': bool(result)
                })
```

```
            except Exception:
                pass

            if result:
                return jsonify({
                    'success': True,
                    'cookies': [result],
                    'method': 'PIR'
                })
            else:
                return jsonify({'error': 'Invalid PIR query'}), 400
        else:
            # Regular retrieval (filtered by domain or all)
            if domain:
                cookies = cookie_store.get_cookies_by_domain(domain)
            else:
                cookies = cookie_store.get_all_cookies()

            # Log retrievals (summarized and per cookie for visibility)
            try:
                for ck in cookies:
                    cookie_logger.log_cookie_retrieved(ck, method='direct')
            except Exception:
                pass

            return jsonify({
                'success': True,
                'cookies': cookies,
                'count': len(cookies),
                'method': 'direct'
            })

    except Exception as e:
        return jsonify({'error': str(e)}), 500
```

simple_pir class :

```
class SimplePIR:
    def __init__(self):
        self.key = get_random_bytes(32)  # AES-256 key

    def encrypt_query(self, index):
        """
        Encrypt the query index using a simple obfuscation
        In production, use proper PIR schemes like SealPIR or SimplePIR
        """
        cipher = AES.new(self.key, AES.MODE_ECB)
        # Pad index to 16 bytes
        padded_index = pad(str(index).encode(), AES.block_size)
        encrypted = cipher.encrypt(padded_index)
        return base64.b64encode(encrypted).decode()

    def process_pir_query(self, encrypted_query, database):
        """
```

```
        Process PIR query - in this simplified version, we decrypt the query
        In production PIR, the server wouldn't learn which item is accessed
        """
        try:
            cipher = AES.new(self.key, AES.MODE_ECB)
            encrypted_bytes = base64.b64decode(encrypted_query)
            decrypted = unpad(cipher.decrypt(encrypted_bytes), AES.block_size)
            index = int(decrypted.decode())

            if 0 <= index < len(database):
                return database[index]
            return None
        except Exception as e:
            print(f"PIR query error: {e}")
            return None

    def get_key_for_client(self):
        """Return the encryption key for the client (extension)"""
        return base64.b64encode(self.key).decode()
```

Browser Extension

apply browser cookies

```
async function postClientLog(type, data) {
  try {
    await fetchWithBackoff(`${PIR_SERVER_URL}/api/logs/client`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ type, data })
    });
  } catch (_) {
    // best effort only
  }
}

// Apply cookies to browser
async function applyCookiesToBrowser(cookies) {
  let successCount = 0;
  let errorCount = 0;

  for (const cookie of cookies) {
    try {
      // Prepare cookie for Chrome API
      const domainNoDot = (cookie.domain || '').startsWith('.') ?
cookie.domain.slice(1) : cookie.domain;
      const cookieDetails = {
        url: `http${cookie.secure ? 's' : ''}://${domainNoDot}${cookie.path}`,
        name: cookie.name,
        value: cookie.value,
        domain: cookie.domain,
        path: cookie.path,
        secure: cookie.secure,
        httpOnly: cookie.httpOnly,
        sameSite: cookie.sameSite ? cookie.sameSite.toLowerCase() : 'lax'
```

```
      };

      // Set expiration if provided
      if (cookie.expirationDate) {
        cookieDetails.expirationDate = cookie.expirationDate;
      }

      await chrome.cookies.set(cookieDetails);
      successCount++;
    } catch (error) {
      console.error(`Error setting cookie ${cookie.name}:`, error);
      errorCount++;
    }
  }

  return { successCount, errorCount };
}
```

manifest.json

```
{
    "manifest_version": 3,
    "name": "PIR Cookie Manager",
    "version": "1.0.0",
    "description": "Private Information Retrieval based cookie storage and retrieval",
    "permissions": [
      "cookies",
      "storage",
      "tabs",
      "activeTab",
      "webNavigation",
      "idle"
    ],
    "host_permissions": [
      "<all_urls>"
    ],
    "background": {
    "service_worker": "background.js"
    },
  "content_security_policy": {
    "extension_pages": "script-src 'self' 'wasm-unsafe-eval'; object-src 'self';
worker-src 'self'"
  },
  "web_accessible_resources": [
    {
      "resources": [
        "models/*",
        "libs/*"
      ],
      "matches": ["<all_urls>"]
    }
  ],
  "options_page": "dashboard.html",
```

```
    "action": {
      "default_popup": "popup.html",
      "default_icon": {
        "16": "icons/icon16.png",
        "48": "icons/icon48.png",
        "128": "icons/icon128.png"
      }
    },
    "icons": {
      "16": "icons/icon16.png",
      "48": "icons/icon48.png",
      "128": "icons/icon128.png"
    }
  }
```

Remote server classification

```
async function classifyCookieRemote(cookie) {
  const response = await fetchWithBackoff(`${PIR_SERVER_URL}/api/cookies/classify`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ cookie })
  });
  const data = await response.json();
  if (!data || data.success !== true) {
    throw new Error(data?.error || 'Remote classification failed');
  }
  return {
    category: data.category,
    confidence: data.confidence,
    class_index: data.class_index,
    should_store: !!data.should_store,
  };
}
```